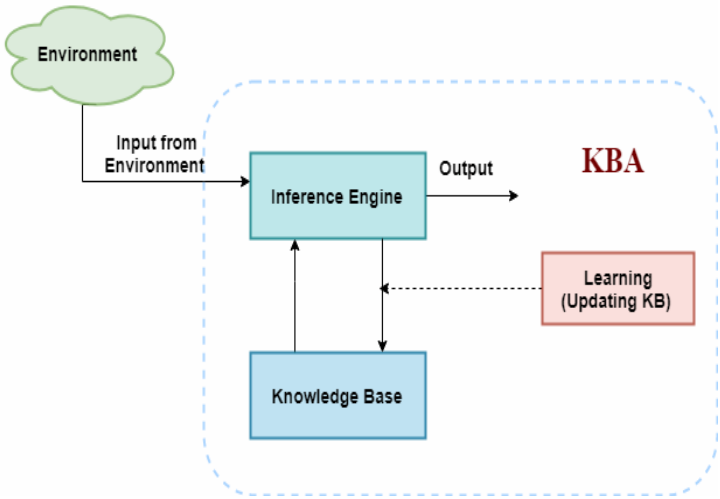
UNIT-3

Knowledge Representation

3.1 Knowledge-Based Agents

An intelligent agent needs knowledge about the real world for taking decisions and reasoning to act efficiently. Knowledge-based agents are those agents who have the capability of maintaining an internal state of knowledge, reason over that knowledge, update their knowledge after observations and take actions. These agents can represent the world with some formal representation and act intelligently.

- Knowledge-based agents are composed of two main parts:
  - ❖ Knowledge-base and
  - ❖ Inference system.



This diagram is representing a generalized architecture for a knowledge-based agent. The knowledge-based agent (KBA) take input from the environment by perceiving the environment. The input is taken by the inference engine of the agent and which also communicate with KB to decide as per the knowledge store in KB. The learning element of KBA regularly updates the KB by learning new knowledge

Knowledge base: Knowledge-base is a central component of a knowledge-based agent, it is also known as KB. It is a collection of sentences (here 'sentence' is a technical term and it is not identical to sentence in English). These sentences are expressed in a language which is called a knowledge representation language. The Knowledge-base of KBA stores fact about the world.

Why use a knowledge base?

Knowledge-base is required for updating knowledge for an agent to learn with experiences and take action as per the knowledge.

Inference system

Inference means deriving new sentences from old. Inference system allows us to add a new sentence to the knowledge base. A sentence is a proposition about the world. Inference system applies logical rules to the KB to deduce new information.

Inference system generates new facts so that an agent can update the KB. An inference system works mainly in two rules which are given as:

Forward chaining

Backward chaining

Operations Performed by KBA

Following are three operations which are performed by KBA in order to show the intelligent behavior:

1. **TELL:** This operation tells the knowledge base what it perceives from the environment.
2. **ASK:** This operation asks the knowledge base what action it should perform.
3. **Perform:** It performs the selected action.

A generic knowledge-based agent:

Following is the structure outline of a generic knowledge-based agents program:

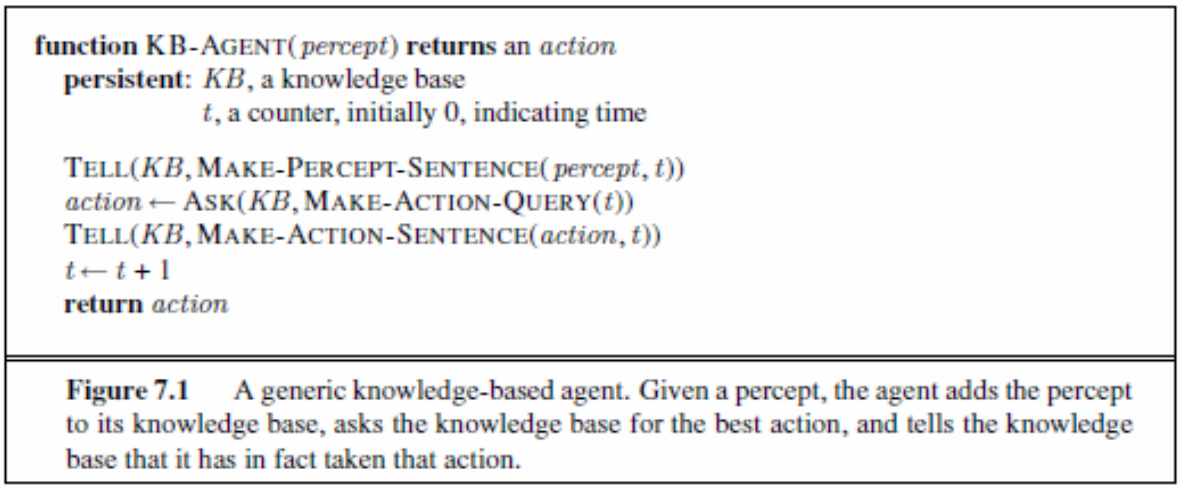


Figure 7.1 A generic knowledge-based agent. Given a percept, the agent adds the percept to its knowledge base, asks the knowledge base for the best action, and tells the knowledge base that it has in fact taken that action.

The knowledge-based agent takes percept as input and returns an action as output. The agent maintains the knowledge base, KB, and it initially has some background knowledge of the real world. It also has a counter to indicate the time for the whole process, and this counter is initialized with zero.

Each time when the function is called, it performs its three operations:

- Firstly it TELLs the KB what it perceives.
- Secondly, it asks KB what action it should take
- Third agent program TELLS the KB that which action was chosen.

The **MAKE-PERCEPT-SENTENCE** generates a sentence as setting that the agent perceived the given percept at the given time. The **MAKE-ACTION-QUERY** generates a sentence to ask which action should be done at the current time. The **MAKE-ACTION-SENTENCE** generates a sentence which asserts that the chosen action was executed.

Various levels of knowledge-based agent:

A knowledge-based agent can be viewed at different levels which are given below:

1**. Knowledge level**

Knowledge level is the first level of knowledge-based agent, and in this level, we need to specify what the agent knows, and what the agent goals are. With these specifications, we can fix its behavior. For example, suppose an automated taxi agent needs to go from a station A to station B, and he knows the way from A to B, so this comes at the knowledge level.

2. Logical level:

At this level, we understand that how the knowledge representation of knowledge is stored. At this level, sentences are encoded into different logics. At the logical level, an encoding of knowledge into logical sentences occurs. At the logical level we can expect to the automated taxi agent to reach to the destination B.

3. **Implementation level:**

This is the physical representation of logic and knowledge. At the implementation level agent perform actions as per logical and knowledge level. At this level, an automated taxi agent actually implement his knowledge and logic so that he can reach to the destination.

Approaches to designing a knowledge-based agent:

There are mainly two approaches to build a knowledge-based agent:

1. Declarative approach: We can create a knowledge-based agent by initializing with an empty knowledge base and telling the agent all the sentences with which we want to start with. This approach is called Declarative approach.

2. Procedural approach: In the procedural approach, we directly encode desired behavior as a program code. Which means we just need to write a program that already encodes the desired behavior or agent.

However, in the real world, a successful agent can be built by combining both, declarative and procedural approaches, and declarative knowledge can often be compiled into more efficient procedural code.

Wumpus World

The wumpus world is a cave consisting of rooms connected by passageways. Lurking somewhere in the cave is the terrible wumpus, a beast that eats anyone who enters its room. The wumpus can be shot by an agent, but the agent has only one arrow.
Some rooms contain bottomless pits that will trap anyone who wanders into these rooms (except for the wumpus, which is too big to fall in). The only mitigating feature of this bleak environment is the possibility of finding a heap of gold.

PEAS Environment for Wumpus World

Performance Measure: +1000 for exiting with gold, −1000 for dying, −1 per action, −10 per arrow use.

Environment:
4×4 grid, start at [1,1] facing right. Gold & Wumpus randomly placed (not at start). Each non-start square has 20% chance of a pit.

Actuators:
Move Forward, Turn Left/Right, Grab gold, Shoot arrow (once), Climb (only at [1,1]). Death if entering pit or live Wumpus.

Sensors:

- Stench near Wumpus.
- Breeze near pit.
- Glitter on gold square.
- Bump if hit wall.
- Scream when Wumpus dies.

For example, if there is a stench and a breeze, but no glitter, bump, or scream, the agent program will get [Stench, Breeze, None, None, None].

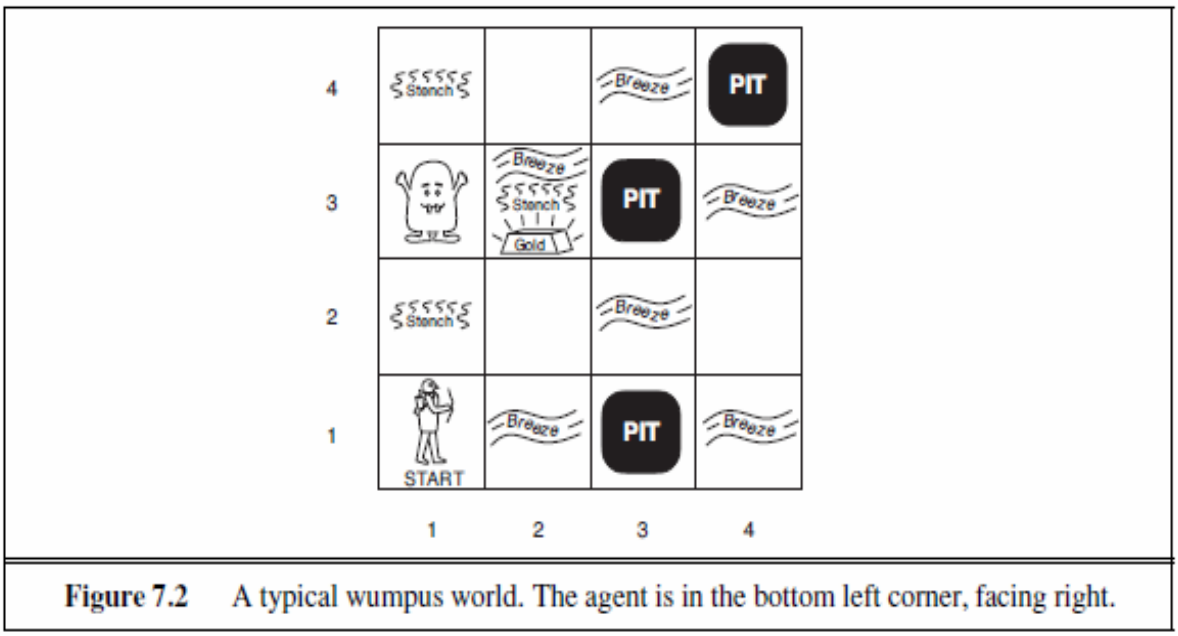


Figure 7.2 A typical wumpus world. The agent is in the bottom left corner, facing right.

How it works:

The grid is a 4×4 world. The agent starts at the bottom-left corner (1,1), facing right. There are hazards:

- PIT: deadly pit (if agent steps in, it dies).
- Wumpus: a monster that kills the agent if they enter its cell.

There is Gold: the agent's goal is to find and grab it, then exit.

Percepts (Clues):

- Stench: in any square adjacent to the Wumpus.
- Breeze: in any square adjacent to a pit.

- Gold glitter: in the square containing the gold.

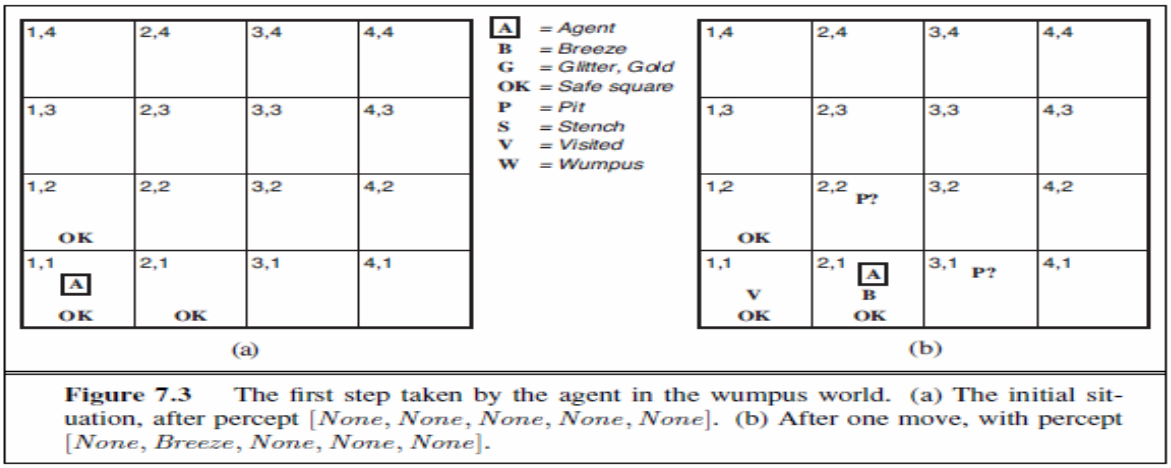


Figure 7.3 The first step taken by the agent in the wumpus world. (a) The initial situation, after percept [*None*, *None*, *None*, *None*, *None*]. (b) After one move, with percept [*None*, *Breeze*, *None*, *None*, *None*].

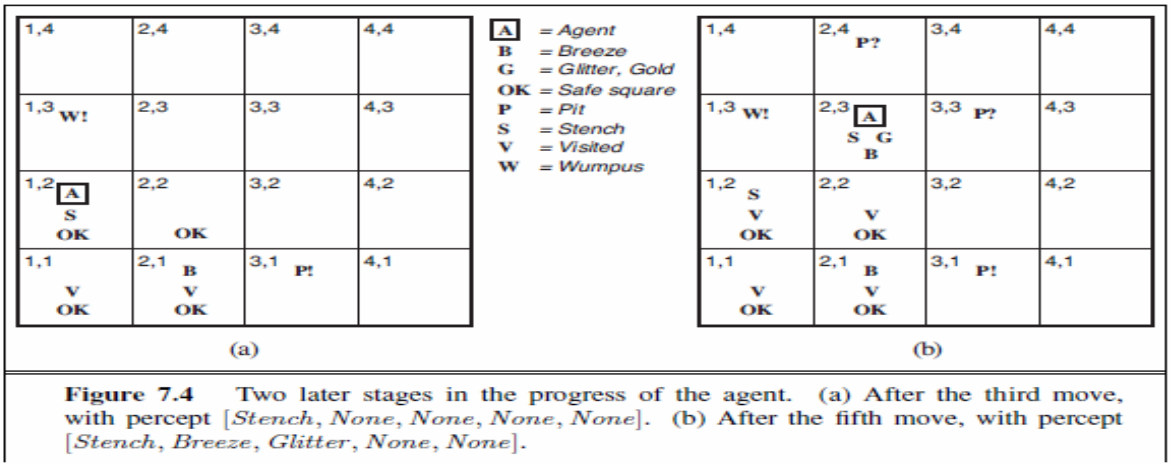


Figure 7.4 Two later stages in the progress of the agent. (a) After the third move, with percept [*Stench*, *None*, *None*, *None*, *None*]. (b) After the fifth move, with percept [*Stench*, *Breeze*, *Glitter*, *None*, *None*].

Start: Agent at [1,1], knows it's safe (A, OK).Percept: [None, None, None, None, None] → Neighboring squares [1,2] and [2,1] are safe.

Move to [2,1]:

- Percept: Breeze (B) → Pit nearby in [2,2] or [3,1] (marked P?).
- Only other safe unvisited square is [1,2]. Agent returns to [1,1] and moves to [1,2].

At [1,2]:

- Percept: Stench (S) → Wumpus nearby.
- Cannot be in [1,1] or [2,2] → Wumpus is in [1,3] (W!).
- No breeze here → no pit in [2,2].
- Earlier info said pit is in [2,2] or [3,1] → must be in [3,1].

Inference: [2,2] is safe → Agent can move there, then to [2,3]. At [2,3]:Percept: Glitter → Gold found! Grab it and return to start.

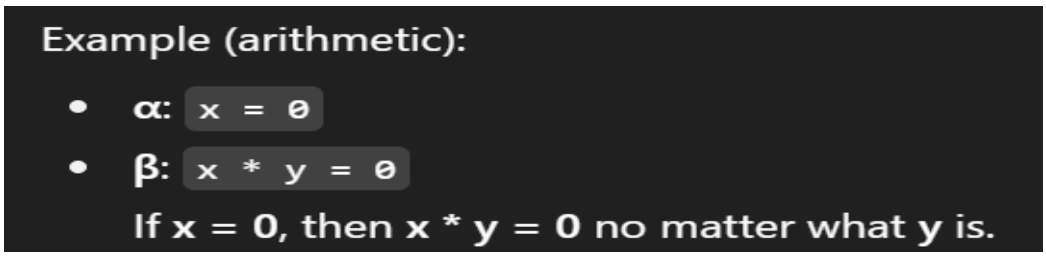
3.2 Logic

These sentences are expressed according to the syntax of the representation language, which specifies all the sentences that are well formed. The notion of syntax is clear enough in ordinary arithmetic: "x + y = 4" is a well-formed sentence, whereas "x4y+ =" is not. logic must also define the semantics or meaning of sentences. The semantics defines the truth of each sentence with respect to each possible world. For example, the semantics for arithmetic specifies that the sentence "x + y =4" is true in a world where x is 2 and y is 2, but false in a world where x is 1 and y is 1. In standard logics, every sentence must be either true or false in each possible world—there is no "in between."

- When being precise, we use the term model instead of "possible world."
    - ❖ Possible world: Think of it as a real situation the agent could be in.
    - ❖ Model: A mathematical description that decides whether each statement is true or false.
- Example: If a possible world has x men and y women at a table,
    - ❖ The statement x + y = 4 is true if there are exactly 4 people.
    - ❖ Formally, models are all possible assignments of numbers to x and y.
- A model m satisfies a sentence α if α is true in that model. We write M(α) to mean "the set of all models where α is true." Entailment means that one sentence logically follows from another. In math notation:

  $\alpha \models \beta$
- This means: If α is true, then β must also be true. Sentence α entails sentence β if, in every model where α is true, β is also true. Formally: $\alpha \models \beta$ if and only if $M(\alpha) \subseteq M(\beta)$ This means: all models that make α true also make β true. α is a stronger statement than β because it rules out more possibilities.

Example (arithmetic):



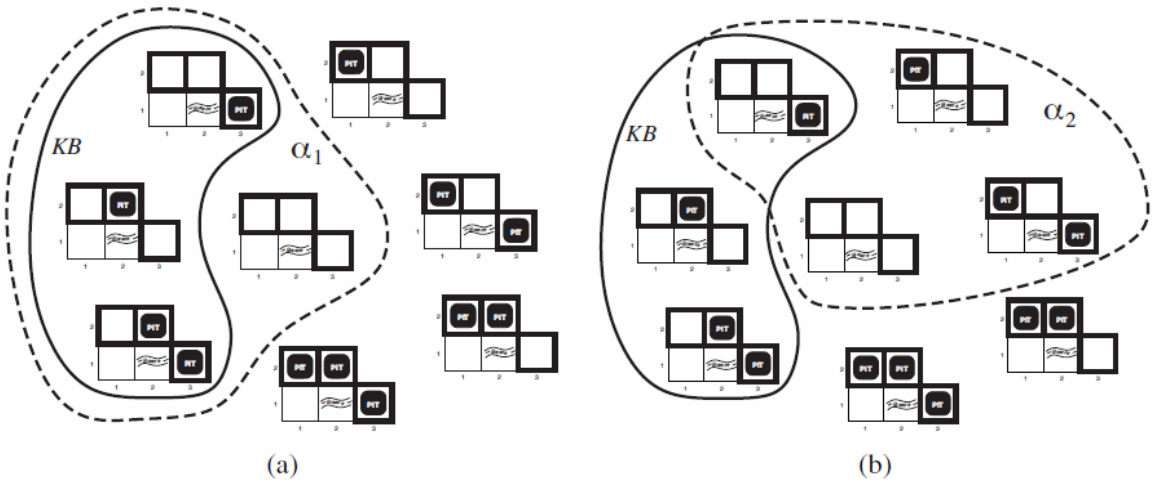


Figure 7.5 Possible models for the presence of pits in squares [1,2], [2,2], and [3,1]. The KB corresponding to the observations of nothing in [1,1] and a breeze in [2,1] is shown by the solid line. (a) Dotted line shows models of α_1 (no pit in [1,2]). (b) Dotted line shows models of α_2 (no pit in [2,2]).

- In the Wumpus World example (Figure 7.3b), the agent starts at [1,1] with no breeze and detects a breeze at [2,1]. This information, plus the rules of the game, forms the knowledge base (KB). The agent wants to know if [1,2], [2,2], and [3,1] have pits. Each square may or may not have a pit, so there are $2^3 = 8$ possible models (worlds). Only 3 models match the KB(solid lines in Figure 7.5).
- Two statements are tested:
  - ❑ α : "No pit in [1,2]"
  - ❑ α : "No pit in [2,2]"
- From the models: α is true in all models where KB is true → KB entails α . α is not true when KB is true → KB does not entail α . This shows entailment (what logically follows from KB) and inference (the process of finding it).
- Model checking tests entailment by listing all possible models and checking if α is true in all models where KB is true. Soundness: An inference method is sound if it only produces correct conclusions (model checking is sound). Completeness: A method is complete if it can find every conclusion that logically follows from KB.

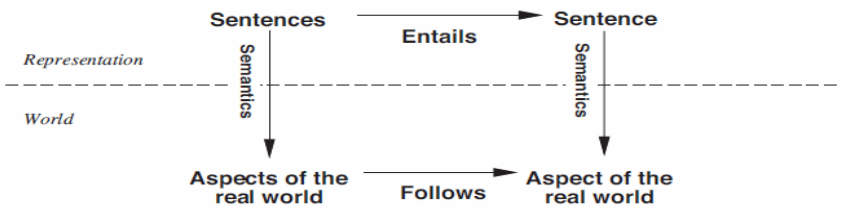


Figure 7.6 Sentences are physical configurations of the agent, and reasoning is a process of constructing new physical configurations from old ones. Logical reasoning should ensure that the new configurations represent aspects of the world that actually follow from the aspects that the old configurations represent.

The final issue to consider is grounding—the connection between logical reasoning processes and the real environment in which the agent exists. In particular, how do we know that KB is true in the real world? (After all, KB is just "syntax" inside the agent's head.). An agent's sensors connect its knowledge to the real world. For example, in the Wumpus World, the agent's smell sensor detects odors and adds a "smell" statement to its knowledge base. This means that whenever the sensor detects a smell, the knowledge base reflects something true in the real world. However, not all knowledge comes directly from sensors. Some, like "wumpuses cause smells in nearby squares," are general rules learned from experience. Learning isn't perfect—there could be exceptions (like wumpuses bathing on February 29). So, the knowledge base might sometimes be wrong, but with good learning methods, it can still be mostly accurate.

3.3 Propositional Logic

- A very simple but powerful logic called Propositional Logic.

3.3.1 Syntax

- The syntax of propositional logic defines the allowable sentences. The atomic sentences consist of a single proposition symbol. Each such symbol stands for a proposition that can be true or false. We use symbols that start with an uppercase letter and may contain other letters or subscripts, for example: P, Q, R, $W_{1,3}$ and North.
    - $W_{1,3}$- Wumpus is in [1,3].
- There are two proposition symbols with fixed meanings: True is the always-true proposition and False is the always-false proposition.
- Complex sentences are constructed from simpler sentences, using parentheses and logical connectives. There are five connectives in common use:
- ¬(not). A sentence such as $\neg W_{1,3}$ is called the negation of $W_{1,3}$. A literal is either an atomic sentence (a positive literal) or a negated atomic sentence (a negative literal).
- ∧ (and). A sentence whose main connective is ∧, such as $W_{1,3} \wedge P_{3,1}$, is called a conjunction; its parts are the conjuncts. (The ∧ looks like an "A" for "And.")
- ∨ (or). A sentence using ∨, such as $(W_{1,3} \wedge P_{3,1}) \vee W_{2,2}$, is a disjunction of the disjuncts $(W_{1,3} \wedge P_{3,1})$ and $W_{2,2}$.
- ⇒ (implies). A sentence such as $(W_{1,3} \wedge P_{3,1}) \Rightarrow \neg W_{2,2}$ is called an implication (or conditional). Its premise or antecedent is $(W_{1,3} \wedge P_{3,1})$, and its conclusion or consequent is $\neg W_{2,2}$. Implications are also known as rules or if–then statements. The implication symbol is sometimes written in other books as ⊃ or →.
- ⇔ (if and only if). The sentence $W_{1,3} \Leftrightarrow \neg W_{2,2}$ is a biconditional. Some other books write this as ≡.
- A sentence with several operators can be parsed the grammar in multiple ways. To eliminate the ambiguity we define a precedence for each operator. The "not" operator

(¬) has the highest precedence, which means that in the sentence ¬A ∧ B the ¬ binds most tightly, giving us the equivalent of (¬A)∧B rather than ¬(A∧B).

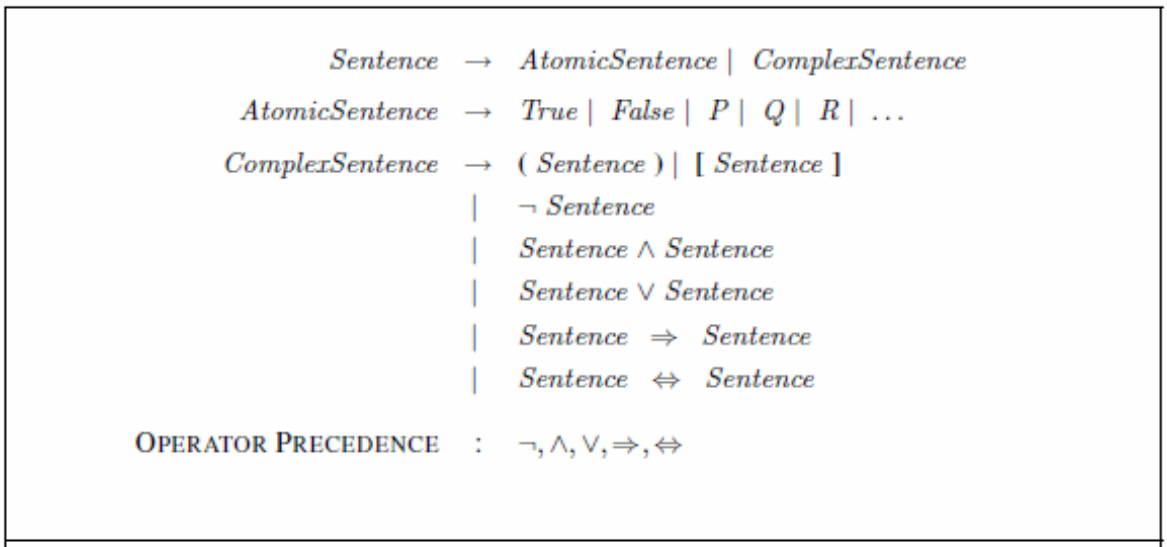


Figure 7.7 A BNF (Backus–Naur Form) grammar of sentences in propositional logic, along with operator precedences, from highest to lowest.

3.3.2 Semantics

The semantics defines the rules for determining the truth of a sentence with respect to a particular model. In propositional logic, a model simply fixes the truth value—true or false—for every proposition symbol. For example, if the sentences in the knowledge base make use of the proposition symbols $P_{1,2}$, $P_{2,2}$, and $P_{3,1}$, then one possible model is

$$m_1 = \{P_{1,2} = \text{false}, P_{2,2} = \text{false}, P_{3,1} = \text{true}\}$$

With three proposition symbols, there are $2^3 = 8$ possible models—exactly those depicted. Notice, however, that the models are purely mathematical objects with no necessary connection to wumpus worlds. $P_{1,2}$ is just a symbol; it might mean "there is a pit in [1,2]" or "I'm in Paris today and tomorrow." In propositional logic, semantics define how to determine a sentence's truth value from a model, using recursion. Truth values are first assigned to atomic sentences, then computed for complex sentences using the five connectives.

- ❑ True is true in every model and False is false in every model.
- ❑ The truth value of every other proposition symbol must be specified directly in the model. For example, in the model m1 given earlier, $P_{1,2}$ is false.

For complex sentences, we have five rules, which hold for any subsentences P and Q in any model m (here "iff" means "if and only if"):

- ❖ ¬P is true iff P is false in m.
- ❖ P ∧ Q is true iff both P and Q are true in m.
- ❖ P ∨ Q is true iff either P or Q is true in m.

- ❖ P ⇒ Q is true unless P is true and Q is false in m.
- ❖ P ⇔ Q is true iff P and Q are both true or both false in m

The rules can also be expressed with truth tables that specify the truth value of a complex sentence for each possible assignment of truth values to its components

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	*false*	*true*	*false*	*false*	*true*	*true*
false	*true*	*true*	*false*	*true*	*true*	*false*
true	*false*	*false*	*false*	*true*	*false*	*false*
true	*true*	*false*	*true*	*true*	*true*	*true*

Figure 7.8 Truth tables for the five logical connectives. To use the table to compute, for example, the value of $P \vee Q$ when P is true and Q is false, first look on the left for the row where P is *true* and Q is *false* (the third row). Then look in that row under the $P \vee Q$ column to see the result: *true*.

The main point of possible confusion is that P ∨Q is true when P is true or Q is true or both. A different connective, called "exclusive or" ("xor" for short), yields false when both disjuncts are true. In propositional logic, "P ⇒ Q" does not imply causation or relevance—it's true whenever P is false or both P and Q are true. The only false case is when P is true and Q is false. The biconditional "P ⇔ Q" means both "P ⇒ Q" and "Q ⇒ P" are true ("if and only if").

Example:

- states a square is breezy exactly when a neighboring square has a pit.

3.3.3 A simple knowledge base

We focus first on the immutable aspects of the wumpus world, leaving the mutable aspects for a later section. For now, we need the following symbols for each [x, y] location:

- ❖ $P_{x,y}$ is true if there is a pit in [x, y].
- ❖ $W_{x,y}$ is true if there is a wumpus in [x, y], dead or alive.
- ❖ $B_{x,y}$ is true if the agent perceives a breeze in [x, y].
- ❖ $S_{x,y}$ is true if the agent perceives a stench in [x, y].

The sentences we write will suffice to derive $\neg P_{1,2}$ (there is no pit in [1,2]), as was done informally. We label each sentence R_i so that we can refer to them:

- There is no pit in [1,1]:

  $R_1 : \neg P_{1,1}$

A square is breezy if and only if there is a pit in a neighboring square. This has to be stated for each square; for now, we include just the relevant squares:

$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

$R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$

The preceding sentences are true in all wumpus worlds. Now we include the breeze percepts for the first two squares visited in the specific world the agent is in, leading up to the situation in Figure 7.3(b).

$R_4 : \neg B_{1,1}$

$R_5 : B_{2,1}$

3.3.4 A simple inference procedure

Our first algorithm for inference is a model-checking approach that is a direct implementation of the definition of entailment: enumerate the models, and check that α is true in every model in which KB is true.

Models are assignments of true or false to every proposition symbol. Returning to our wumpus-world example, the relevant proposition symbols are $B_{1,1}$, $B_{2,1}$, $P_{1,1}$, $P_{1,2}$, $P_{2,1}$, $P_{2,2}$, and $P_{3,1}$. With seven symbols, there are 2^7 =128 possible models; in three of these, KB is true (Figure 7.9).

In those three models, $\neg P_{1,2}$ is true, hence there is no pit in [1,2]. On the other hand, $P_{2,2}$ is true in two of the three models and false in one, so we cannot yet tell whether there is a pit in [2,2].

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	*false*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*false*	*false*
false	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*false*	*true*	*false*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	*true*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*false*	*true*	*true*	*false*
false	*true*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*false*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*true*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	*true*	*true*	*true*	*true*	*true*	*true*	*false*	*true*	*true*	*false*	*true*	*false*

Figure 7.9 A truth table constructed for the knowledge base given in the text. *KB* is true if R_1 through R_5 are true, which occurs in just 3 of the 128 rows (the ones underlined in the right-hand column). In all 3 rows, $P_{1,2}$ is false, so there is no pit in [1,2]. On the other hand, there might (or might not) be a pit in [2,2].

```
function TT-ENTAILS?(KB, α) returns true or false
  inputs: KB, the knowledge base, a sentence in propositional logic
          α, the query, a sentence in propositional logic

  symbols ← a list of the proposition symbols in KB and α
  return TT-CHECK-ALL(KB, α, symbols, { })

function TT-CHECK-ALL(KB, α, symbols, model) returns true or false
  if EMPTY?(symbols) then
      if PL-TRUE?(KB, model) then return PL-TRUE?(α, model)
      else return true // when KB is false, always return true
  else do
      P ← FIRST(symbols)
      rest ← REST(symbols)
      return (TT-CHECK-ALL(KB, α, rest, model ∪ {P = true})
              and
              TT-CHECK-ALL(KB, α, rest, model ∪ {P = false }))
```

Figure 7.10 A truth-table enumeration algorithm for deciding propositional entailment. (TT stands for truth table.) PL-TRUE? returns *true* if a sentence holds within a model. The variable *model* represents a partial model—an assignment to some of the symbols. The keyword "**and**" is used here as a logical operation on its two arguments, returning *true* or *false*.

$$
\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}
$$

Figure 7.11 Standard logical equivalences. The symbols α, β, and γ stand for arbitrary sentences of propositional logic.

3.4 Syntax and semantics of First Order Logic

3.4.1 Models for first-order logic

Models in logic describe possible worlds by linking vocabulary (symbols) to objects and their relations. Domain = set of objects in the model (must be nonempty). A model = domain , relations , functions, which lets us decide truth of first-order logic sentences.

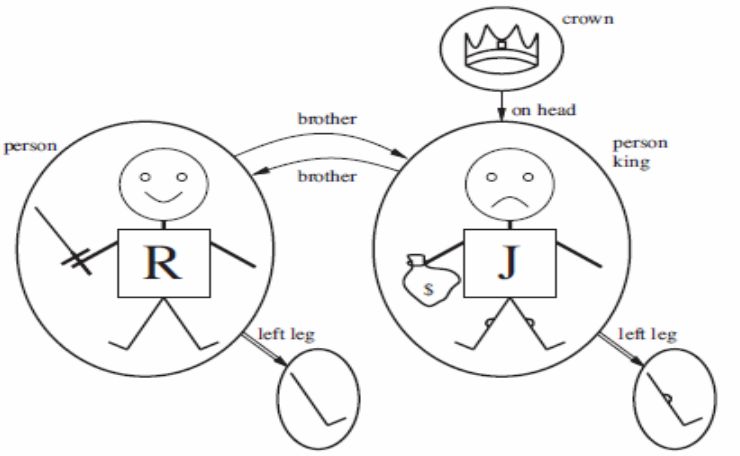


Figure 8.2 A model containing five objects, two binary relations, three unary relations (indicated by labels on the objects), and one unary function, left-leg.

- Example model (Fig 8.2) has 5 objects: Richard, John, Richard's left leg, John's left leg, crown.

Binary relations:

- Brother = {⟨Richard, John⟩, ⟨John, Richard⟩}
- On head = {⟨crown, John⟩}

Unary relations (properties):

- Person: Richard, John
- King: John
- Crown: crown

Unary function:

- Left leg: Richard → Richard's left leg; John → John's left leg

Richard and John are brothers. Formally speaking, a relation TUPLE is just the set of tuples of objects that are related. (A tuple is a collection of objects arranged in a fixed order and is written with angle brackets surrounding the objects.) Thus, the brotherhood relation in this model is the set

- { Richard the Lionheart, King John, King John, Richard the Lionheart }

For example, each person has one left leg, so the model has a unary "left leg" function that includes the following mappings:

- {Richard the Lionheart}→ Richard's left leg
- {King John} → John's left leg .

Strictly speaking, models in first-order logic require total functions, that is, there must be a value for every input tuple.

3.4.2 Symbols and interpretations

a)What a model is in FOL:

- A model consists of:
  - Objects → the elements of the domain.
  - Interpretation → mapping of:
    - ✓ Constant symbols → to specific objects.
    - ✓ Predicate symbols → to relations among objects.
    - ✓ Function symbols → to functions on objects.

b) Interpretations

- Different models are possible because the mapping is flexible.
- Example:
  - Richard → Richard the Lionheart.
  - John → King John.
  - Brother → brotherhood relation.
  - OnHead → relation between crown and King John.
  - Person, King, Crown → sets of objects.
  - LeftLeg → a mapping from a person to their left leg.
- Other mappings are possible (e.g., Richard → crown, John → a leg).
- Objects may:
  - Have no names (unnamed objects exist in the domain).
  - Have multiple names (two constants can refer to the same object).

c) Variety of Models

- Models vary by:
  - Number of objects (from 1 to infinity).
  - How constants are mapped to objects.
- Example: with two constants but only one object, both constants must refer to the same object.
- Even with more objects, constants can still point to the same object.

d) Why model checking is hard

- In propositional logic, models are finite and enumerable, so entailment can be checked directly.
- In first-order logic:

- The number of possible models is unbounded.
- Even if restricted to a small number of objects, combinations grow very large.
- Example: with ≤ 6 objects, there are 137,506,194,466 possible models (from Fig. 8.4).

- Hence, entailment cannot be decided by brute-force enumeration.
- A model in FOL = domain (objects) + interpretation (mapping constants → objects, predicates → relations, functions → functions).

$$
\begin{array}{rcl}
\textit{Sentence} & \rightarrow & \textit{AtomicSentence} \mid \textit{ComplexSentence} \\
\textit{AtomicSentence} & \rightarrow & \textit{Predicate} \mid \textit{Predicate}(\textit{Term}, \ldots) \mid \textit{Term} = \textit{Term} \\
\textit{ComplexSentence} & \rightarrow & (\ \textit{Sentence}\ ) \mid [\ \textit{Sentence}\ ] \\
 & \mid & \neg\ \textit{Sentence} \\
 & \mid & \textit{Sentence} \wedge \textit{Sentence} \\
 & \mid & \textit{Sentence} \vee \textit{Sentence} \\
 & \mid & \textit{Sentence} \Rightarrow \textit{Sentence} \\
 & \mid & \textit{Sentence} \Leftrightarrow \textit{Sentence} \\
 & \mid & \textit{Quantifier}\ \textit{Variable}, \ldots\ \textit{Sentence} \\
\textit{Term} & \rightarrow & \textit{Function}(\textit{Term}, \ldots) \\
 & \mid & \textit{Constant} \\
 & \mid & \textit{Variable} \\
\textit{Quantifier} & \rightarrow & \forall \mid \exists \\
\textit{Constant} & \rightarrow & A \mid X_1 \mid \textit{John} \mid \cdots \\
\textit{Variable} & \rightarrow & a \mid x \mid s \mid \cdots \\
\textit{Predicate} & \rightarrow & \textit{True} \mid \textit{False} \mid \textit{After} \mid \textit{Loves} \mid \textit{Raining} \mid \cdots \\
\textit{Function} & \rightarrow & \textit{Mother} \mid \textit{LeftLeg} \mid \cdots \\
\text{OPERATOR PRECEDENCE} & : & \neg, =, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{array}
$$

Figure 8.3 The syntax of first-order logic with equality, specified in Backus–Naur form (see page 1060 if you are not familiar with this notation). Operator precedences are specified, from highest to lowest. The precedence of quantifiers is such that a quantifier holds over everything to the right of it.

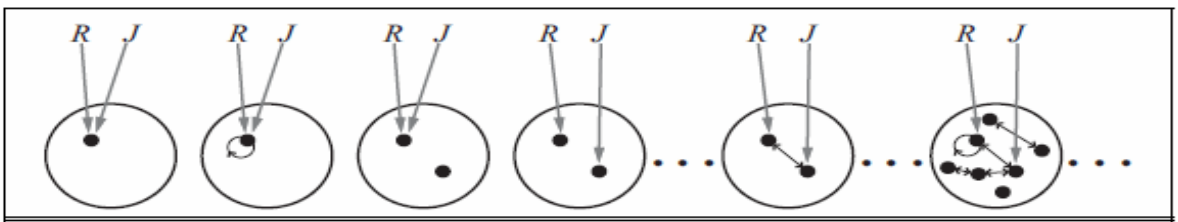


Figure 8.4 Some members of the set of all models for a language with two constant symbols, R and J, and one binary relation symbol. The interpretation of each constant symbol is shown by a gray arrow. Within each model, the related objects are connected by arrows.

3.4.3 Terms

Term: A term is any logical expression that refers to an object in the domain.

Examples:

- Constant symbols (e.g., John, Richard).Function-based expressions (e.g., LeftLeg(John)). Function-based expressions (e.g., LeftLeg(John)).
- Function Symbols are used because
  - Not every object has its own name (constant).
  - Function symbols let us describe objects without introducing new constants.

- **Example:** instead of naming "John's left leg," we write LeftLeg(John).

Complex Term

- A complex term = function symbol and arguments (which are themselves terms).

  General form: f(t1, …, tn).

- Example: LeftLeg(John) → the left leg of John.
- Function symbols are not subroutines (unlike in programming).
  - They don't "compute" anything.
  - They just name an object in the domain, as defined by the model's interpretation.
  - Example: LeftLeg is a mapping already fixed by the model.

Semantics of terms:

- If f refers to a function F in the model:
  - $t_1, \ldots, t_n \rightarrow$ refer to objects $d_1, \ldots, d_n$ in the domain.
  - Then $f(t_1, \ldots, t_n)$ refers to the object $F(d_1, \ldots, d_n)$.
- Example:
    - If John → King John.
    - If LeftLeg → mapping from people to their left legs.
    - Then LeftLeg(John) → King John's left leg.
- A term = name for an object.
- Constants are simple terms; function symbols create complex terms. Interpretation tells us exactly which object each term refers to.

3.4.4 Atomic sentences

- Atomic Sentence (or atom for short) : predicate symbol optionally followed by a parenthesized list of terms, such as

  Brother (Richard , John).

- This states, under the intended interpretation given earlier, that Richard the Lionheart is the brother of King John.6 Atomic sentences can have complex terms as arguments. Thus,

  Married(Father (Richard),Mother (John))

- states that Richard the Lionheart's father is married to King John's mother (again, under a suitable interpretation).

3.4.5 Complex sentences

- We can use logical connectives to construct more complex sentences, with the same syntax and semantics as in propositional calculus.
  - ¬Brother (LeftLeg(Richard), John)
  - Brother (Richard , John) ∧ Brother (John,Richard)
  - King(Richard) ∨ King(John)
  - ¬King(Richard) ⇒ King(John) .

3.4.6 Quantifiers

First-order logic contains two standard quantifiers, called universal and existential.

Universal quantification (∀)

In first-order logic, universal quantification allows us to express general rules like "All kings are persons," written as ∀x King(x) ⇒ Person(x). This means that for every object x, if x is a king, then x is a person. A variable (like x) can stand alone as a term or appear inside a function (e.g., LeftLeg(x)). A ground term is a term with no variables. The sentence ∀x P is true in a model if P holds under all possible assignments of x to objects in the domain. For example, with objects {Richard, John, Richard's left leg, John's left leg, crown}, the rule expands into separate implications for each object.

Only King John satisfies the premise "is a king," so the rule asserts that he is a person. For non-kings like legs or the crown, the premise is false, making the implication true without saying anything about them. A common mistake is to write ∀x King(x) ∧ Person(x), which wrongly asserts that every object is both a king and a person. The correct use of implication (⇒) ensures the rule applies only to actual kings. Implication (⇒) is true whenever something is not a king, so it only "checks" kings. Conjunction (∧) wrongly forces everything to be both a king and a person.

King(x)	Person(x)	King(x) ⇒ Person(x)	King(x) ∧ Person(x)
T	T	T ✅ (correct: king is a person)	T ✅
T	F	F ❌ (wrong: king but not a person)	F ❌
F	T	T ✅ (since not a king, rule holds vacuously)	F ❌ (forces non-kings to be kings too)
F	F	T ✅ (since not a king, rule holds vacuously)	F ❌

Existential quantification (∃)

The existential quantifier (∃) is used to state that something is true for at least one object in the domain. For example, ∃x Crown(x) ∧ OnHead(x, John) means "There exists an x such that x is a crown and x is on John's head." This sentence is true in a model if at least one object satisfies both conditions. In the example, the crown itself satisfies them, so the

sentence is true. If John wore two crowns, the sentence would still be true, since it only requires at least one. Just as implication (⇒) works naturally with ∀, conjunction (∧) works naturally with ∃. If we instead wrote ∃x Crown(x) ⇒ OnHead(x, John), the meaning becomes too weak. This is because an implication is true whenever the premise is false, so the sentence would hold as soon as we find any object that is not a crown—something almost always available in the domain. Thus, the correct way to express existential statements is with ∧, not ⇒.

- ∀ → general rules, usually with ⇒.
- ∃ → specific existence, usually with ∧.

Nested quantifiers let us combine multiple quantifiers in a single sentence.

- If the quantifiers are the same type, we can combine them
  - Example: ∀x ∀y Brother(x, y) ⇒ Sibling(x, y) ("Brothers are siblings").
  - Equivalent shorthand: ∀x, y Sibling(x, y) ⇔ Sibling(y, x) (siblinghood is symmetric).
- With mixed quantifiers, order matters:
  - ∀x ∃y Loves(x, y) → everyone loves someone.
  - ∃y ∀x Loves(x, y) → there is someone loved by everyone.
- Parentheses clarify scope: ∀x (∃y Loves(x, y)) vs. ∃y (∀x Loves(x, y)).
- Variables are bound by their closest quantifier. For example,
  - ∀x (Crown(x) ∨ (∃x Brother(Richard, x))) really means
  - ∀x (Crown(x) ∨ (∃z Brother(Richard, z))).
- To avoid confusion, always use different variable names when nesting quantifiers.

2. Existential Quantifier (∃x Crown(x) ∧ OnHead(x, John))

Crown(x)	OnHead(x, John)	Crown(x) ∧ OnHead(x, John)	Notes
T	T	✅ True	A crown is on John's head.
T	F	❌ False	Crown not on head.
F	T	❌ False	On head but not a crown.
F	F	❌ False	Neither.

👉 With ∃, **at least one true case** is enough.

Connections between ∀ and ∃

- The universal (∀) and existential (∃) quantifiers are connected through negation, much like ∧ and ∨ in propositional logic. For example:

- $\forall x \neg Likes(x, Parsnips) \equiv \neg\exists x\ Likes(x, Parsnips)$ ("everyone dislikes parsnips" ≡ "no one likes parsnips").
- $\forall x\ Likes(x, IceCream) \equiv \neg\exists x \neg Likes(x, IceCream)$ ("everyone likes ice cream" ≡ "there is no one who doesn't").
- These follow De Morgan's rules for quantifiers:
  - $\forall x \neg P \equiv \neg\exists x\ P$ $\quad\quad \neg(P \vee Q) \equiv \neg P \wedge \neg Q$
  - $\neg\forall x\ P \equiv \exists x \neg P$ $\quad\quad \neg(P \wedge Q) \equiv \neg P \vee \neg Q$
  - $\forall x\ P \equiv \neg\exists x \neg P$ $\quad\quad P \wedge Q \equiv \neg(\neg P \vee \neg Q)$
  - $\exists x\ P \equiv \neg\forall x \neg P$ $\quad\quad P \vee Q \equiv \neg(\neg P \wedge \neg Q)$
- Thus, either quantifier could be defined in terms of the other, but both are kept for clarity and readability.

3.4.7 Equality

In first-order logic, we can form atomic sentences using the equality symbol (=), which asserts that two terms refer to the same object. For example, Father(John) = Henry states that John's father is Henry. Equality can also be negated to show that two terms are different. For instance, to say Richard has at least two brothers, we write:$\exists x, y$ Brother(x, Richard) ∧ Brother(y, Richard) ∧ ¬(x = y). Without the inequality, the statement could be satisfied even if Richard had only one brother, since both variables might refer to the same person. The shorthand $x \neq y$ is often used instead of ¬(x = y).

3.4.8 An alternative semantics?

Simply writing Brother(John, Richard) ∧ Brother(Geoffrey, Richard) does not fully capture the fact that Richard's brothers are exactly John and Geoffrey.

- We must also assert that John ≠ Geoffrey and that no one else is Richard's brother:
- Brother(John, Richard) ∧ Brother(Geoffrey, Richard) ∧ John ≠ Geoffrey ∧ $\forall x$ (Brother(x, Richard) ⇒ (x = John ∨ x = Geoffrey)).
- This is more cumbersome than natural language. To simplify, database semantics is often used, which assumes:
  - Unique-names assumption → each constant refers to a distinct object.
  - Closed-world assumption → any fact not known to be true is false.
  - Domain closure → the domain has only the objects named by constants.
- Under these rules, the simpler sentence suffices to mean that Richard's only brothers are John and Geoffrey. Database semantics reduces the number of possible models and works well when we know all the objects and facts, but in general reasoning, standard first-order semantics is preferred despite being more verbose.

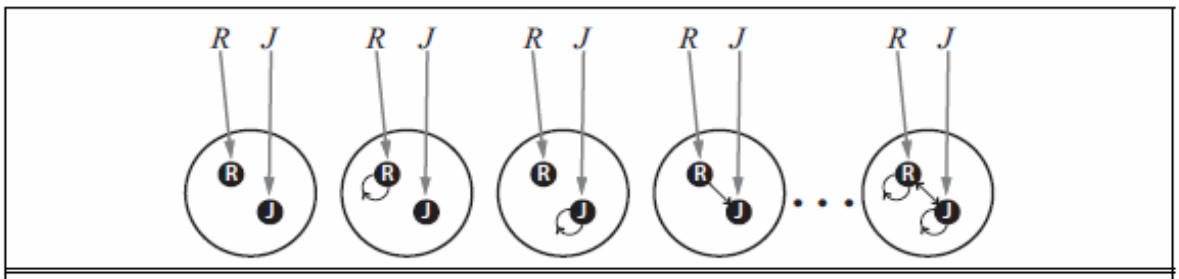


Figure 8.5 Some members of the set of all models for a language with two constant symbols, R and J, and one binary relation symbol, under database semantics. The interpretation of the constant symbols is fixed, and there is a distinct object for each constant symbol.

3.5 Inference Vs First Order Inference

Modern logical inference starts with simple rules that eliminate quantifiers, producing quantifier-free sentences. This allows conversion of a first-order knowledge base into propositional logic, enabling the use of propositional inference. The next step introduces a shortcut: inference methods that work directly on first-order sentences.

3.5.1 Inference rules for quantifiers

Suppose our knowledge base contains the standard folkloric axiom stating that all greedy kings are evil:

$$\forall x \; King(x) \wedge Greedy(x) \Rightarrow Evil(x)$$

Then it seems quite permissible to infer any of the following sentences:

- King(John) ∧ Greedy(John) ⇒ Evil(John)
- King(Richard) ∧ Greedy(Richard) ⇒ Evil(Richard)
- King(Father (John)) ∧ Greedy(Father (John)) ⇒ Evil(Father (John))

Universal Instantiation

The rule of Universal Instantiation (UI for short) says that we can infer any sentence obtained by substituting a ground term (a term without variables) for the variable. Let SUBST(θ,α) denote the result of applying the substitution θ to the sentence α. Then the rule is written $\frac{\forall v \;\; \alpha}{\text{SUBST}(\{v/g\}, \alpha)}$ for any variable v and ground term g. For example, the three sentences given earlier are obtained with the substitutions {x/John}, {x/Richard}, and {x/Father (John)}.

Existential Instantiation

In the rule for Existential Instantiation, the variable is replaced by a single new constant symbol. The formal statement is as follows: for any sentence α, variable v, and constant symbol k that does not appear elsewhere in the knowledge base,

$$\frac{\exists v \;\; \alpha}{\text{SUBST}(\{v/k\}, \alpha)} .$$

For example, from the sentence

$$\exists x \text{ Crown}(x) \land \text{OnHead}(x, \text{John})$$

we can infer the sentence

$$\text{Crown}(C_1) \land \text{OnHead}(C_1, \text{John})$$

As long as C_1 does not appear elsewhere in the knowledge base. Basically, the existential sentence says there is some object satisfying a condition, and applying the existential instantiation rule just gives a name to that object. Of course, that name must not already belong to another object. Mathematics provides a nice example: suppose we discover that there is a number that is a little bigger than 2.71828 and that satisfies the equation d(x^y)/ dy = x^y for x. We can give this number a name, such as e, but it would be a mistake to give it the name of an existing object, such as π.

In logic, the new name is called a **Skolem constant**. Existential Instantiation is a special case of a more general process called **Skolemization**

Here's a **comparison table of Universal Instantiation (UI) vs Existential Instantiation (EI):**

Feature	Universal Instantiation (UI)	Existential Instantiation (EI)
Definition	Replace a variable in ∀x α with any ground term	Replace a variable in ∃x α with a **new constant** (Skolem constant)
Application	Can be applied **many times** with different substitutions	Applied **once**, then the existential sentence is discarded
Example	∀x [King(x) ∧ Greedy(x) ⇒ Evil(x)] → King(John) ∧ Greedy(John) ⇒ Evil(John)	∃x [Crown(x) ∧ OnHead(x, John)] → Crown(C1) ∧ OnHead(C1, John)
Constant used	Any existing ground term	A **fresh symbol** not appearing elsewhere (Skolem constant)
Logical effect	Produces multiple possible consequences	Gives a "name" to the unknown object
Equivalence	Maintains logical equivale	New KB not logically equivalent, but **inferentially equivalent**

3.5.2 Reduction to Propositional Inference

The first idea is that, just as an existentially quantified sentence can be replaced by one instantiation, a universally quantified sentence can be replaced by the set of all possible instantiations. For example, suppose our knowledge base contains just the sentences

- ∀ x King(x) ∧ Greedy(x) ⇒ Evil(x)
- King(John)
- Greedy(John)
- Brother (Richard, John)

Then we apply UI to the first sentence using all possible ground-term substitutions from the vocabulary of the knowledge base—in this case, {x/John} and {x/Richard}. We obtain

- King(John) ∧ Greedy(John) ⇒ Evil(John)
- King(Richard) ∧ Greedy(Richard) ⇒ Evil(Richard)

and we discard the universally quantified sentence. Every first-order knowledge base and query can be propositionalized in such a way that entailment is preserved. Thus, we have a complete decision procedure for entailment ... or perhaps not. There is a problem: when the knowledge base includes a function symbol, the set of possible ground-term substitutions is infinite!

For example, if the knowledge base mentions the Father symbol, then infinitely many nested terms such as Father (Father (Father (John))) can be constructed. Fortunately, there is a famous theorem due to Jacques Herbrand (1930) to the effect that if a sentence is entailed by the original, first-order knowledge base, then there is a proof involving just a finite subset of the propositionalized knowledge base. Since any such subset has a maximum depth of nesting among its ground terms, we can find the subset by first generating all the instantiations with constant symbols (Richard and John), then all terms of depth 1 (Father (Richard) and Father (John)), then all terms of depth 2, and so on, until we are able to construct a propositional proof of the entailed sentence.

3.6 Unification and Lifting

The propositionalization approach (used before the 1960s) was inefficient because it generated unnecessary instances

King(Richard)∧Greedy(Richard)⇒Evil(Richard)

even when the query was only about John.

For example, to infer Evil(John) from:

- ∀x(King(x)∧Greedy(x)⇒Evil(x))
- King(John),Greedy(John)

A human sees the conclusion as obvious, and modern methods aim to make it just as obvious for a computer.

Note: Old propositionalization was inefficient; direct inference makes results like Evil(John) straightforward.

3.6.1 A first-order inference rule

The inference that John is evil—that is, that {x/John} solves the query Evil(x)—works like this: to use the rule that greedy kings are evil, find some x such that x is a king and x is greedy, and then infer that this x is evil. More generally, if there is some substitution θ that makes each of the conjuncts of the premise of the implication identical to sentences already in the knowledge base, then we can assert the conclusion of the implication, after applying θ.

In this case, the substitution θ = {x/John} achieves that aim. Suppose that instead of knowing Greedy(John), we know that everyone is greedy: ∀ y Greedy(y). Since John is included in "everyone," this means Greedy(John). So, combined with King(John), we can still infer Evil(John).

In this case, applying the substitution {x/John, y/John} to the implication premises King(x) and Greedy(x) and the knowledge-base sentences King(John) and Greedy(y) will make them identical. Thus, we can infer the conclusion of the implication.

This inference process can be captured as a single inference rule that we call Generalized Modus Ponens: For atomic sentences P_i, P_i', and q where there is a substitution θ such that SUBST(θ,) = SUBST(θ, pi), for all i,

$$\frac{p_1',\ p_2',\ \ldots,\ p_n',\ (p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q)}{\text{SUBST}(\theta, q)}$$

There are n+1 premises to this rule: the n atomic sentences P_i' and the one implication. The conclusion is the result of applying the substitution θ to the consequent q. For our example:

$$\begin{array}{ll} p_1' \text{ is } King(John) & p_1 \text{ is } King(x) \\ p_2' \text{ is } Greedy(y) & p_2 \text{ is } Greedy(x) \\ \theta \text{ is } \{x/John, y/John\} & q \text{ is } Evil(x) \\ \text{SUBST}(\theta, q) \text{ is } Evil(John)\ . & \end{array}$$

First, we observe that, for any sentence p (whose variables are assumed to be universally quantified) and for any substitution θ,

p |= SUBST(θ, p)

holds by Universal Instantiation. It holds in particular for a θ that satisfies the conditions of the Generalized Modus Ponens rule. Thus, from

we can infer

$$\text{SUBST}(\theta, p_1') \wedge \ldots \wedge \text{SUBST}(\theta, p_n')$$

and from the implication $p_1', \ldots, p_n'$ we can infer

$$\text{SUBST}(\theta, p_1) \wedge \ldots \wedge \text{SUBST}(\theta, p_n) \Rightarrow \text{SUBST}(\theta, q)$$

- Now, θ in Generalized Modus Ponens is defined so that
- For all *i*; therefore the first of these two sentences matches the premise of the second exactly.
- Hence, SUBST(θ, q) follows by Modus Ponens.
- Generalized Modus Ponens is a lifted version of Modus Ponens. it raises Modus Ponens from ground (variable-free) propositional logic to first-order logic.

3.6.2 Unification

Lifted inference rules require finding substitutions that make different logical expressions look identical. This process is called unification and is a key component of all first-order

inference algorithms. The UNIFY algorithm takes two sentences and returns a unifier for them if one exists:

$$\text{UNIFY}(p, q) = \theta \text{ where } \text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$$

finding all sentences in the knowledge base that unify with Knows(John, x). Here are the results of unification with four different sentences that might be in the knowledge base:

$$\begin{aligned}
&\text{UNIFY}(Knows(John, x),\ Knows(John, Jane)) = \{x/Jane\} \\
&\text{UNIFY}(Knows(John, x),\ Knows(y, Bill)) = \{x/Bill, y/John\} \\
&\text{UNIFY}(Knows(John, x),\ Knows(y, Mother(y))) = \{y/John, x/Mother(John)\} \\
&\text{UNIFY}(Knows(John, x),\ Knows(x, Elizabeth)) = fail\ .
\end{aligned}$$

The last unification fails because x cannot take on the values John and Elizabeth at the same time. Now, remember that Knows(x, Elizabeth) means “Everyone knows Elizabeth,”. so we should be able to infer that John knows Elizabeth. The problem arises only because the two sentences happen to use the same variable name, x.

The problem can be avoided by standardizing apart one of the two sentences being unified, which means renaming its variables to avoid name clashes. For example, we can rename x in Knows(x, Elizabeth) to x_{17} (a new variable name) without changing its meaning. Now the unification will work:

$$\text{UNIFY}(Knows(John, x),\ Knows(x_{17}, Elizabeth)) = \{x/Elizabeth, x_{17}/John\}$$

3.6.3 Storage and retrieval

Underlying the TELL and ASK functions used to inform and interrogate a knowledge base are the more primitive STORE and FETCH functions. STORE(s) stores a sentence s into the knowledge base and FETCH(q) returns all unifiers such that the query q unifies with some sentence in the knowledge base.

The problem we used to illustrate unification—finding all facts that unify with Knows(John, x)—is an instance of FETCHing. The simplest way to implement STORE and FETCH is to keep all the facts in one long list and unify each query against every element of the list.

```
function UNIFY(x, y, θ) returns a substitution to make x and y identical
  inputs: x, a variable, constant, list, or compound expression
          y, a variable, constant, list, or compound expression
          θ, the substitution built up so far (optional, defaults to empty)

  if θ = failure then return failure
  else if x = y then return θ
  else if VARIABLE?(x) then return UNIFY-VAR(x, y, θ)
  else if VARIABLE?(y) then return UNIFY-VAR(y, x, θ)
  else if COMPOUND?(x) and COMPOUND?(y) then
      return UNIFY(x.ARGS, y.ARGS, UNIFY(x.OP, y.OP, θ))
  else if LIST?(x) and LIST?(y) then
      return UNIFY(x.REST, y.REST, UNIFY(x.FIRST, y.FIRST, θ))
  else return failure
```

```
function UNIFY-VAR(var, x, θ) returns a substitution

  if {var/val} ∈ θ then return UNIFY(val, x, θ)
  else if {x/val} ∈ θ then return UNIFY(var, val, θ)
  else if OCCUR-CHECK?(var, x) then return failure
  else return add {var/x} to θ
```

Figure 9.1 The unification algorithm. The algorithm works by comparing the structures of the inputs, element by element. The substitution θ that is the argument to UNIFY is built up along the way and is used to make sure that later comparisons are consistent with bindings that were established earlier. In a compound expression such as $F(A, B)$, the OP field picks out the function symbol F and the ARGS field picks out the argument list (A, B).

- Query: Knows(John, x)
- Sentence in KB: Brother(Richard, John)

Knows and Brother are totally different function names. So, no matter what substitution we try, these two will never match. Trying to unify them is useless and inefficient. We can avoid such unifications by indexing the facts in the knowledge base.

predicate indexing:

puts all the Knows facts in one bucket and all the Brother facts in another. The buckets can be stored in a hash table for efficient access. Predicate indexing helps when there are many predicates but few clauses for each. But if a single predicate (like Employs(x, y)) has millions of employers and tens of millions of employees, the bucket becomes too large.

Answering a query like Employs(x, Richard) would still require scanning the entire bucket, making it inefficient.

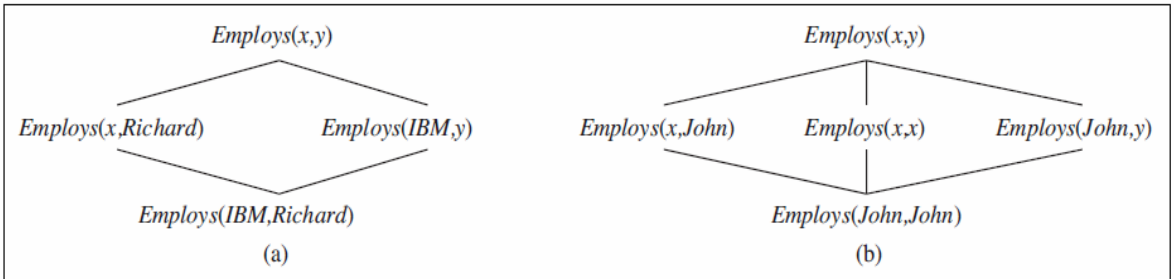


Figure 9.2 (a) The subsumption lattice whose lowest node is $Employs(IBM, Richard)$. (b) The subsumption lattice for the sentence $Employs(John, John)$.

Predicate indexing alone is inefficient for large predicates like Employs(x, y) with millions of entries. Efficiency improves by indexing facts on predicate + argument(s) (e.g., by employer or by employee) using combined keys.

Example: Employs(IBM, Richard) supports queries like:

Employs(IBM, Richard) → Does IBM employ Richard?

Employs(x, Richard) → Who employs Richard?

Employs(IBM, y) → Whom does IBM employ?

Employs(x, y) → Who employs whom?

These queries form a **subsumption lattice** where each child is derived by a substitution, and the most general unifier finds common descendants. Lattices are manageable for few nodes, but grow exponentially ($O(2^n)$) with arguments or function symbols. Too many indices become costly → so use fixed indexing policies (predicate + arguments) or adaptive indexing depending on queries. In AI systems (small data), indexing is solved; in commercial databases (billions of facts), indexing remains a major challenge.

3.7 Forward Chaining

Forward Chaining start with the atomic sentences in the knowledge base and apply Modus Ponens in the forward direction, adding new atomic sentences, until no further inferences can be made. For first-order definite clauses (e.g., Situation ⇒ Response), it's efficient and well-suited for systems that react to new information.

3.7.1 First-order definite clauses

First-order definite clauses are like propositional ones: disjunctions with exactly one positive literal. They can be atomic (e.g., King(John)) or implications (e.g., King(x) ∧ Greedy(x) ⇒ Evil(x)).

$$King(x) \wedge Greedy(x) \Rightarrow Evil(x) .$$
$$King(John) .$$
$$Greedy(y) .$$

Variables in them are universally quantified (assumed ∀).
Not all knowledge bases can be written as definite clauses, but many can due to the single-positive-literal restriction.

Example:

The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by ColonelWest, who is American.

We will prove that West is a criminal. First, we will represent these facts as first-order definite clauses. The next section shows how the forward-chaining algorithm solves the problem.

3.7.2 A simple forward-chaining algorithm

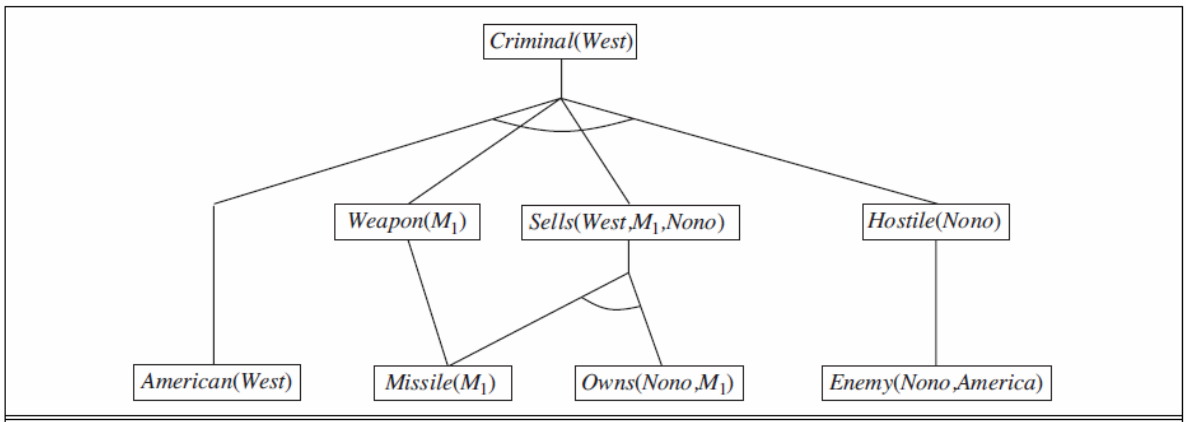


Figure 9.4 The proof tree generated by forward chaining on the crime example. The initial facts appear at the bottom level, facts inferred on the first iteration in the middle level, and facts inferred on the second iteration at the top level.

- It is a crime for an American to sell weapons to hostile nations":

  American(x) ∧Weapon(y) ∧ Sells(x, y, z) ∧ Hostile(z) ⇒ Criminal (x) .

- "Nono . . . has some missiles." The sentence ∃ x Owns(Nono, x)∧Missile(x) is transformed into two definite clauses by Existential Instantiation, introducing a new constant M1:

Owns(Nono,M1)

Missile(M1)

- "All of its missiles were sold to it by Colonel West":

  Missile(x) ∧ Owns(Nono, x) ⇒ Sells(West, x, Nono) .

- We will also need to know that missiles are weapons:

  Missile(x) ⇒ Weapon(x)

- and we must know that an enemy of America counts as "hostile":

Enemy(x,America) ⇒ Hostile(x)

- "West, who is American . . .":

  American(West) .

"The country Nono, an enemy of America . . .":

  Enemy(Nono,America) .

Datalog is a language of first-order definite clauses without function symbols. It's named for its similarity to relational database statements. The absence of function symbols makes inference simpler and more efficient.

Knowledge base

Facts (initial):

- F1: $American(West)$
- F2: $Enemy(Nono, America)$
- F3: $Owns(Nono, M_1)$
- F4: $Missile(M_1)$

Rules:

- R1: $American(x) \land Weapon(y) \land Sells(x, y, z) \land Hostile(z) \Rightarrow Criminal(x)$
- R2: $Missile(m) \Rightarrow Weapon(m)$
- R3: $Owns(Nono, m) \land Missile(m) \Rightarrow Sells(West, m, Nono)$
- R4: $Enemy(u, America) \Rightarrow Hostile(u)$

Forward chaining (GMP applications)

Iteration 1

- From F4 with R2: unify $m \mapsto M_1$
  $\Rightarrow Weapon(M_1)$.
- From F3 & F4 with R3: unify $m \mapsto M_1$
  $\Rightarrow Sells(West, M_1, Nono)$.
- From F2 with R4: unify $u \mapsto Nono$
  $\Rightarrow Hostile(Nono)$.

Iteration 2

- Premises of R1 are now all satisfied:
  - $American(West)$ (F1)
  - $Weapon(M_1)$ (iter. 1)
  - $Sells(West, M_1, Nono)$ (iter. 1)
  - $Hostile(Nono)$ (iter. 1)
- Unifier $\theta = \{x \mapsto West,\ y \mapsto M_1,\ z \mapsto Nono\}$.
- Apply R1 with θ:
  $\Rightarrow Criminal(West)$.

No other new (non-renamed) facts arise, and the query is proved:

$$\boxed{Criminal(West)}$$

```
function FOL-FC-ASK(KB, α) returns a substitution or false
  inputs: KB, the knowledge base, a set of first-order definite clauses
          α, the query, an atomic sentence
  local variables: new, the new sentences inferred on each iteration

  repeat until new is empty
      new ← { }
      for each rule in KB do
          (p1 ∧ ... ∧ pn ⇒ q) ← STANDARDIZE-VARIABLES(rule)
          for each θ such that SUBST(θ, p1 ∧ ... ∧ pn) = SUBST(θ, p'1 ∧ ... ∧ p'n)
                      for some p'1, ..., p'n in KB
              q' ← SUBST(θ, q)
              if q' does not unify with some sentence already in KB or new then
                  add q' to new
                  φ ← UNIFY(q', α)
                  if φ is not fail then return φ
      add new to KB
  return false
```

Figure 9.3 A conceptually straightforward, but very inefficient, forward-chaining algorithm. On each iteration, it adds to KB all the atomic sentences that can be inferred in one step from the implication sentences and the atomic sentences already in KB. The function STANDARDIZE-VARIABLES replaces all variables in its arguments with new ones that have not been used before.

- Let k be the maximum arity (number of arguments) of any predicate, p be the number of predicates, and n be the number of constant symbols.
- For general definite clauses with function symbols, FOL-FC-ASK can generate infinitely many new facts, so we need to be more careful. For the case in which an answer to the query sentence q is entailed, we must appeal to Herbrand's theorem to establish that the algorithm will find a proof.
- If the query has no answer, the algorithm could fail to terminate in some cases. For example, if the knowledge base includes the Peano axioms

  NatNum(0)

  ∀ n NatNum(n) ⇒ NatNum(S(n)) ,

- then forward chaining adds NatNum(S(0)), NatNum(S(S(0))), NatNum(S(S(S(0)))), and so on.

3.7.3 Efficient Forward Chaining:

The forward-chaining algorithm is designed for ease of understanding rather than for efficiency of operation.

First, the "inner loop" of the algorithm involves finding all possible unifiers such that the premise of a rule unifies with a suitable set of facts in the knowledge base. This is often called pattern matching and can be very expensive.

Second, the algorithm rechecks every rule on every iteration to see whether its premises are satisfied, even if very few additions are made to the knowledge base on each iteration.

Finally, the algorithm might generate many facts that are irrelevant to the goal. We address each of these issues in turn.

The conjunct ordering problem arises when matching rule premises (with multiple conditions) against facts in the knowledge base. The goal is to decide in which order to test the conjuncts to minimize cost. For example, in

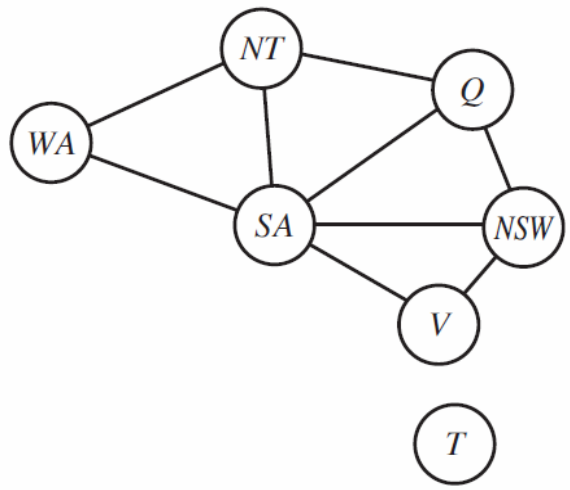
Missile(x) ∧ Owns(Nono, x) ⇒ Sells(West, x, Nono)

we can either:

- Find all objects Nono owns, then check which are missiles, or
- Find all missiles, then check which ones Nono owns.

The second is cheaper if missiles are fewer.

Finding the optimal ordering is NP-hard, but heuristics like MRV (minimum-remaining-values) help by testing the most restrictive conjunct first.



(a)

$$Diff(wa, nt) \wedge Diff(wa, sa) \wedge$$
$$Diff(nt, q) \wedge Diff(nt, sa) \wedge$$
$$Diff(q, nsw) \wedge Diff(q, sa) \wedge$$
$$Diff(nsw, v) \wedge Diff(nsw, sa) \wedge$$
$$Diff(v, sa) \Rightarrow Colorable()$$
$$Diff(Red, Blue) \quad Diff(Red, Green)$$
$$Diff(Green, Red) \quad Diff(Green, Blue)$$
$$Diff(Blue, Red) \quad Diff(Blue, Green)$$

(b)

Figure 9.5 (a) Constraint graph for coloring the map of Australia. (b) The map-coloring CSP expressed as a single definite clause. Each map region is represented as a variable whose value can be one of the constants *Red*, *Green* or *Blue*.

- Pattern matching in forward chaining is closely related to CSPs: each conjunct is like a constraint. Since CSPs (and 3-SAT) are NP-hard, rule matching is also NP-hard in general.
- But it's manageable because:
  - Data complexity is polynomial if rule size and predicate arity are small.
  - Tractable subclasses exist (e.g., tree-structured CSPs → linear-time matching).
  - Redundancy elimination in rule matching can speed things up.
- So, despite worst-case hardness, forward chaining is practical in real-world KBs.

(a) Constraint graph for coloring the map of Australia

- Each node = a region (WA, NT, Q, NSW, V, SA, T).

- Each edge = "must have different colors" constraint between adjacent regions.
- Example: WA is adjacent to NT and SA → so WA ≠ NT and WA ≠ SA.

- This is the graph representation of the CSP (Constraint Satisfaction Problem).

(b) CSP expressed as a single definite clause

- All adjacency constraints are written as Diff(x, y), meaning "region x and region y must have different colors." The big clause is:

- Diff(wa,nt)∧Diff(wa,sa)∧Diff(nt,q)∧Diff(nt,sa)∧Diff(q,nsw)∧Diff(q,sa)∧Diff(nsw,v)∧Diff(nsw,sa)∧Diff(v,sa)⇒Colorable()
- This means: if all neighboring pairs of regions are colored differently, then the map is colorable.
- Ground facts (the allowed color differences)
- Below the clause you see things like:
  - Diff(Red, Blue)
  - Diff(Red, Green)
  - Diff(Green, Blue)etc.
- These are the basic facts saying which color pairs are different. Together, they define what "Diff" actually means.
- So, if wa = Red, then to satisfy Diff(wa, nt), we must have nt = Blue or nt = Green.
- How this works
  - Assign colors (Red, Green, Blue) to each region (variables).
  - Use the Diff facts to enforce constraints.
  - If all Diff constraints are satisfied, forward chaining can infer Colorable().

Incremental forward chaining

Forward chaining can waste effort by repeatedly re-matching rules with already known facts.

Improvements:

- Incremental forward chaining: At iteration t, only check rules involving facts newly inferred at t–1.
- Partial match storage: Keep unfinished matches instead of rebuilding them each time.
- Rate algorithm: Builds a network of rule premises, keeps partial matches, and filters variable bindings efficiently.

- Used in production systems (e.g., XCON, OPS-5) and cognitive architectures (ACT, SOAR), enabling real-time reasoning with millions of rules.

3.8 Backward Chaining

Backward chaining is a logical inference method that starts from a goal (query) and works backward to determine if it can be supported by known facts and rules. It is widely used in logic programming (e.g., Prolog). The method is efficient for answering specific queries but can face disadvantages such as redundant searches or infinite loops. To address this, optimizations and connections with constraint satisfaction problems are explored.

3.8.1 A backward-chaining algorithm

```
function FOL-BC-ASK(KB, query) returns a generator of substitutions
  return FOL-BC-OR(KB, query, { })

generator FOL-BC-OR(KB, goal, θ) yields a substitution
  for each rule (lhs ⇒ rhs) in FETCH-RULES-FOR-GOAL(KB, goal) do
    (lhs, rhs) ← STANDARDIZE-VARIABLES((lhs, rhs))
    for each θ′ in FOL-BC-AND(KB, lhs, UNIFY(rhs, goal, θ)) do
      yield θ′

generator FOL-BC-AND(KB, goals, θ) yields a substitution
  if θ = failure then return
  else if LENGTH(goals) = 0 then yield θ
  else do
    first, rest ← FIRST(goals), REST(goals)
    for each θ′ in FOL-BC-OR(KB, SUBST(θ, first), θ) do
      for each θ″ in FOL-BC-AND(KB, rest, θ′) do
        yield θ″
```

Figure 9.6 A simple backward-chaining algorithm for first-order knowledge bases.

A backward-chaining algorithm

- FOL-BC-ASK(KB, goal) will be proved if the knowledge base contains a clause of the form lhs ⇒ goal, where lhs (left-hand side) is a list of conjuncts. An atomic fact like American(West) is considered as a clause whose lhs is the empty list. Now a query that contains variables might be proved in multiple ways.
- For example, the query Person(x) could be proved with the substitution {x/John} as well as with {x/Richard }. So we implement FOL-BC-ASK as a function that returns multiple times, each time giving one possible result.
- Backward chaining is a kind of AND/OR search—the OR part because the goal query can be proved by any rule in the knowledge base, and the AND part because all the conjuncts in the lhs of a clause must be proved.
- FOL-BC-OR works by fetching all clauses that might unify with the goal, standardizing the variables in the clause to be brand-new variables, and then, if the rhs of the clause does indeed unify with the goal, proving every conjunct in the lhs, using FOL-BC-AND.

- Backward chaining, as we have written it, is clearly a depth-first search algorithm. This means that its space requirements are linear in the size of the proof (neglecting, for now, the space required to accumulate the solutions).
- It also means that backward chaining (unlike forward chaining) suffers from problems with repeated states and incompleteness.

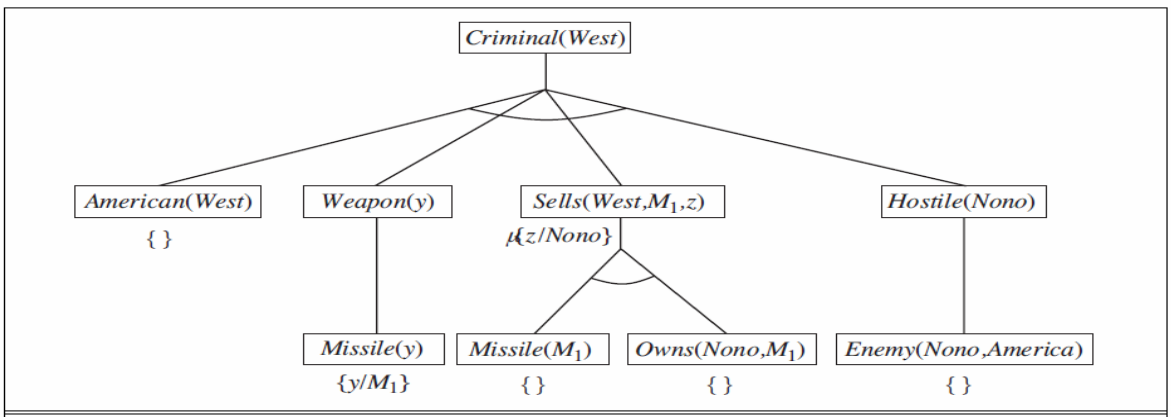


Figure 9.7 Proof tree constructed by backward chaining to prove that West is a criminal. The tree should be read depth first, left to right. To prove *Criminal*(*West*), we have to prove the four conjuncts below it. Some of these are in the knowledge base, and others require further backward chaining. Bindings for each successful unification are shown next to the corresponding subgoal. Note that once one subgoal in a conjunction succeeds, its substitution is applied to subsequent subgoals. Thus, by the time FOL-BC-ASK gets to the last conjunct, originally *Hostile*(*z*), *z* is already bound to *Nono*.

Explanations of the notations on the tree

- Each node is a goal/subgoal.
- Braces like {} next to a leaf mean "a fact was used, no new substitution."
- A brace like { y / M1 } next to a node means that node produced the binding y = M1 by unification.
- μ{z/Nono} near the Sells node in the figure indicates that proving Sells produced the binding z = Nono (often called μ or θ for the substitution). That substitution is then applied to subsequent subgoals in the conjunction.

Example:

Goal

Criminal(West)

- We try to prove this by backward chaining using rule (1). Unify the goal with the head of rule (1):

  Unify Criminal(West) with Criminal(x) → substitution θ = { x / West }.

- So we must prove the body of (1) with x replaced by West:

  American(West) ∧ Weapon(y) ∧ Sells(West, y, z) ∧ Hostile(z)

- The tree in the figure shows these four conjuncts as four children of the root.

Step-by-step proof (depth-first, left-to-right)

1. Prove American(West)

- ❖ This is a fact in the KB (American(West)), so it succeeds immediately.
- ❖ No new substitution: {}.
- ❖ Cumulative substitution so far: { x / West } (from the top).

2. Prove Weapon(y)

- ❖ Use rule Weapon(y) ← Missile(y).
- ❖ Need to prove Missile(y).
- ❖ Missile(M1) is a fact.
- ❖ Unify Missile(y) with Missile(M1) → substitution θ1 = { y / M1 }.
- ❖ So Weapon(y) succeeds with y bound to M1.

Cumulative substitution now: { x/West, y/M1 }.

Prove Sells(West, y, z) with current y = M1

- ❖ After applying y/M1, this subgoal becomes Sells(West, M1, z).
- ❖ Use the rule Sells(x,y,z) ← Missile(y) ∧ Owns(z,y).
- ❖ Unify Sells(West,M1,z) with Sells(x,y,z) → substitution (consistent with earlier ones) gives { x/West, y/M1 } (these match existing bindings).
- ❖ Now we must prove the body: Missile(M1) and Owns(z, M1).
  - Missile(M1) is a fact → succeeds ({}).
  - Owns(z, M1) must match Owns(Nono, M1) → unify gives θ2 = { z / Nono }.
- ❖ So Sells(West,M1,z) succeeds and yields z bound to Nono.

Cumulative substitution now: { x/West, y/M1, z/Nono }.

4. Prove Hostile(z) — but remember z is now bound

- ❖ Because z was bound by the successful proof of Sells(...), the remaining conjunct Hostile(z) is now Hostile(Nono).
- ❖ Use Hostile(z) ← Enemy(z, America). Prove Enemy(Nono, America).
- ❖ Enemy(Nono, America) is a fact → succeeds.

- ➢ So Hostile(z) succeeds under z = Nono.

Result:

- ➢ All four conjuncts succeed under the final composed substitution:

  μ = { x / West, y / M1, z / Nono }.

- ➢ Thus Criminal(West) is proven. The figure's root is true with that substitution.

3.8.2 Logic Programming

- ➢ The ideal is summed up in Robert Kowalski's equation,

- Algorithm = Logic + Control

Prolog Overview

- Logic Programming Language (most widely used).
- **Applications:** rapid prototyping, symbol manipulation (compilers, NLP), expert systems (legal, medical, financial).

Notation:

- **Variables:** Uppercase (e.g., X)
- **Constants:** Lowercase (e.g., john)
- **Clause form:** Head :- Body.
- **Equivalent to:** $A \wedge B \Rightarrow C \leftrightarrow C :- A, B$

Example Rule

criminal(X) :- american(X), weapon(Y), sells(X,Y,Z), hostile(Z).

Means: X is a criminal if X is American, owns a weapon Y, sells it to Z, and Z is hostile.

Lists in Prolog

Notation: [E|L] → list with head E and tail L.

Append Program

append([],Y,Y)

append([A|X],Y,[A|Z]) :- append(X,Y,Z).

Query Example ?- append(X,Y,[1,2]).

$(X,Y) \in \{([],[1,2]),([1],[2]),([1,2],[])\}$

Execution

Search strategy: Depth-first backward chaining.

Special cases:

- Arithmetic: "X is 4+3" → X=7
- "5 is X+Y" → fails
- Built-in predicates: assert/1, retract/1, I/O.
- Occur check omitted → possible unsound inference.
- No infinite recursion check → may loop forever.

3.8.3 Efficient implementation of logic programs

Prolog Execution Modes

Interpreted Mode

Works like FOL-BC-ASK (backward chaining). Uses choice points stack (global structure) instead of manual iteration.

Uses logic variables instead of explicit substitutions.

- ❑ Each variable = either unbound or bound.
- ❑ Trail stack keeps track of bindings for backtracking.

On failure → backtrack to choice point → unbind variables from trail.

Drawback: Each inference step = several thousand machine instructions (slow, because it must always re-match clauses).

Compiled Mode

Program is translated to machine code (or specialized virtual instructions). Avoids repeated clause lookup and unification overhead. Runs much faster than interpreted mode. Behaves as if interpreter has "seen the program before."

Key Idea

- ➢ Interpreted Prolog → flexible, debugging-friendly, but slower.
- ➢ Compiled Prolog → optimized execution, fewer repeated lookups, much faster.

```
procedure APPEND(ax, y, az, continuation)

  trail ← GLOBAL-TRAIL-POINTER()
  if ax = [] and UNIFY(y, az) then CALL(continuation)
  RESET-TRAIL(trail)
  a, x, z ← NEW-VARIABLE(), NEW-VARIABLE(), NEW-VARIABLE()
  if UNIFY(ax, [a | x]) and UNIFY(az, [a | z]) then APPEND(x, y, z, continuation)
```

Figure 9.8 Pseudocode representing the result of compiling the `Append` predicate. The function NEW-VARIABLE returns a new variable, distinct from all other variables used so far. The procedure CALL(*continuation*) continues execution with the specified continuation.

Key ideas:

- Continuations are used to explore multiple solutions (choice points).
- Parallelism can speed execution further:
  - OR-parallelism: explore multiple matching clauses in parallel.
  - AND-parallelism: solve conjuncts in parallel (harder due to shared variables).

3.8.4 Redundant inference and infinite loops

Prolog's weakness comes from its depth-first search:

Problem 1 – Infinite paths:

- The order of rules can cause Prolog to follow an infinite recursion (e.g., path-finding example). This makes it incomplete—it may miss valid solutions.

Problem 2 – Redundant computations: Prolog repeatedly recomputes the same subgoals, leading to exponential work. For graphs, this is the repeated-state problem.

Forward chaining avoids these issues by caching results (dynamic programming), ensuring at most n^2 facts for n nodes.

Fix in backward chaining:

Use memoization (tabled logic programming) → cache and reuse subgoal results.

- Keeps backward chaining's goal-directed search.
- Gains forward chaining's efficiency.
- Makes Prolog complete for Datalog (no missed answers due to infinite loops).

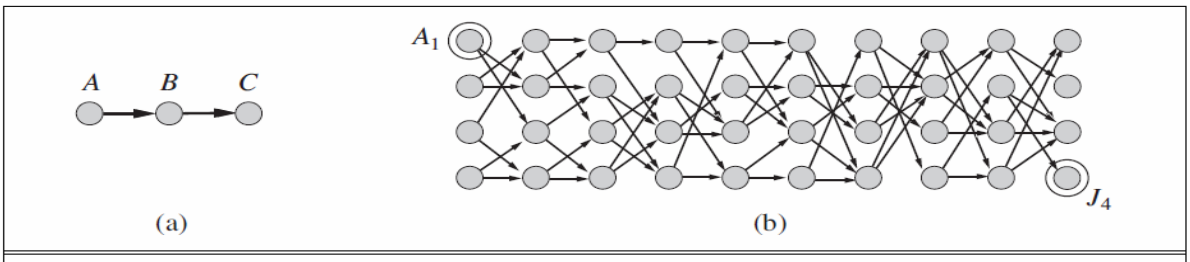


Figure 9.9 (a) Finding a path from A to C can lead Prolog into an infinite loop. (b) A graph in which each node is connected to two random successors in the next layer. Finding a path from A_1 to J_4 requires 877 inferences.

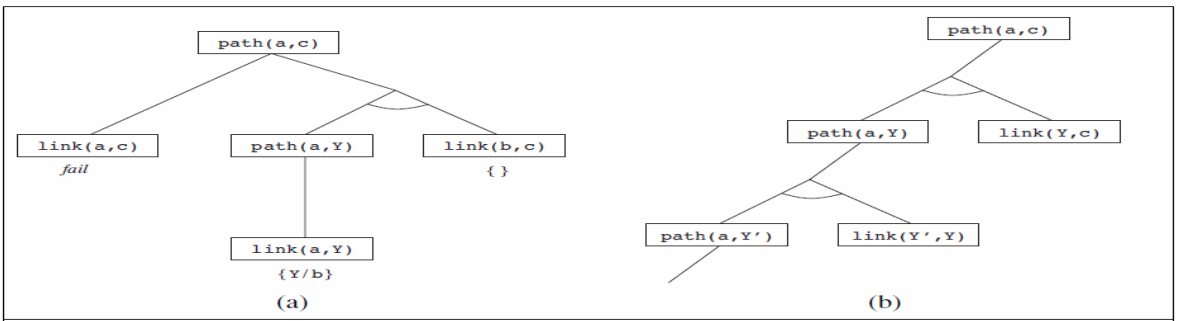


Figure 9.10 (a) Proof that a path exists from A to C. (b) Infinite proof tree generated when the clauses are in the "wrong" order.

3.8.5 Database semantics of Prolog

Prolog follows database semantics:

- Unique Names Assumption (UNA): Each constant/ground term refers to a different object (e.g., CS ≠ EE).
- Closed World Assumption (CWA): Only facts in the knowledge base are true; anything not listed is false.

Example:

Course(CS,101), Course(CS,102), Course(CS,106), Course(EE,101).

→ Exactly four distinct courses (no more, no less).

In FOL, the same assertions would mean:

- There are at least 1 course, maybe infinitely many.
- No guarantee that CS and EE or the numbers are distinct.

To capture Prolog's meaning in FOL, we need completion axioms (explicitly listing all true cases and enforcing equality distinctions).

Standard Prolog solves CSPs using backtracking search, which only works for finite domains.

Example: diff(Q,SA) with 3 colors → 6 solutions.

But queries with unbound variables in infinite domains (e.g., real numbers) fail.

3.8.6 Constraint logic programming

- Extends Prolog by allowing constraints (like inequalities) instead of only bindings
- Consider the following example. We define triangle(X,Y,Z) as a predicate that

holds if the three arguments are numbers that satisfy the triangle inequality:

Example: triangle(X,Y,Z) :-

X>0, Y>0, Z>0, X+Y>=Z, Y+Z>=X, X+Z>=Y.

If we ask Prolog the query triangle(3,4,5), it succeeds. On the other hand, if we ask triangle(3,4,Z), no solution will be found, because the subgoal Z>=0 cannot be handled by Prolog; we can't compare an unbound value to 0.

Advantages of CLP:

- Handles infinite-domain CSPs.
- Uses smarter search (heuristics, backjumping, cutset conditioning, etc.) instead of plain backtracking.
- Combines ideas from CSPs, logic programming, and deductive databases.

Advanced control: Some systems (e.g., MRS) let programmers write metarules to control search order, such as trying goals with fewer variables first.

Prolog = backtracking for finite CSPs; CLP = Prolog + constraints + advanced CSP algorithms, making it far more powerful.

3.9 Resolution

3.9.1 Conjunctive normal form for first-order logic

- As in the propositional case, first-order resolution requires that sentences be in conjunctive normal form (CNF)—that is, a conjunction of clauses, where each clause is a disjunction of literals.
- Literals can contain variables, which are assumed to be universally quantified. For example, the sentence

$$\forall x \text{ American}(x) \land \text{Weapon}(y) \land \text{Sells}(x, y, z) \land \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$$

becomes, in CNF

$$\neg\text{American}(x) \lor \neg\text{Weapon}(y) \lor \neg\text{Sells}(x, y, z) \lor \neg\text{Hostile}(z) \lor \text{Criminal}(x)$$

- Every sentence of first-order logic can be converted into an inferentially equivalent CNF sentence.
- "Everyone who loves all animals is loved by someone," or

$$\forall x [\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)] \Rightarrow [\exists y \text{ Loves}(y, x)] .$$

- The steps are as follows:

Eliminate implications:

$$\forall x [\neg\forall y \neg\text{Animal}(y) \lor \text{Loves}(x, y)] \lor [\exists y \text{ Loves}(y, x)] .$$

Move ¬ inwards: In addition to the usual rules for negated connectives, we need rules for negated quantifiers. Thus, we have

$\neg\forall x\ p$ becomes $\exists x \neg p$

$\neg\exists x\ p$ becomes $\forall x \neg p$

Skolemize: Skolemization is the process of removing existential quantifiers by elimination.

Translate $\exists x\ P(x)$ into $P(A)$, where A is a new constant. However, we can't apply Existential Instantiation to our sentence above because it doesn't match the pattern $\exists v\ \alpha$; only parts of the sentence match the pattern. If we blindly apply the rule to the two matching parts we get

$$\forall x [\text{Animal}(A) \land \neg\text{Loves}(x,A)] \lor \text{Loves}(B, x) ,$$

which has the wrong meaning entirely: it says that everyone either fails to love a particular animal A or is loved by some particular entity B. In fact, our original sentence allows each person to fail to love a different animal or to be loved by a different person. Thus, we want the Skolem entities to depend on x and z:

$$\forall x [\text{Animal}(F(x)) \land \neg\text{Loves}(x, F(x))] \lor \text{Loves}(G(z), x) .$$

Here F and G are Skolem functions. The general rule is that the arguments of the Skolem function are all the universally quantified variables in whose scope the existential quantifier appears. As with Existential Instantiation, the Skolemized sentence is satisfiable exactly when the original sentence is satisfiable.

Drop universal quantifiers: At this point, all remaining variables must be universally quantified. Moreover, the sentence is equivalent to one in which all the universal quantifiers have been moved to the left. We can therefore drop the universal quantifiers:

$$[\text{Animal}(F(x)) \land \neg\text{Loves}(x, F(x))] \lor \text{Loves}(G(z), x) .$$

Distribute ∨ over ∧:

[Animal (F(x)) ∨ Loves(G(z), x)] ∧ [¬Loves(x, F(x)) ∨ Loves(G(z), x)] .

This step may also require flattening out nested conjunctions and disjunctions

3.9.2 The resolution inference rule

Two clauses, which are assumed to be standardized apart so that they share no variables, can be resolved if they contain complementary literals. Propositional literals are complementary if one is the negation of the other; first-order literals are complementary if one unifies with the negation of the other. Thus, we have

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \qquad m_1 \vee \cdots \vee m_n}{\text{SUBST}(\theta, \ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n)}$$

where $\text{UNIFY}(\ell_i, \neg m_j) = \theta$. For example, we can resolve the two clauses

$$[Animal(F(x)) \vee Loves(G(x), x)] \quad \text{and} \quad [\neg Loves(u, v) \vee \neg Kills(u, v)]$$

by eliminating the complementary literals Loves(G(x), x) and ¬Loves(u, v), with unifier

θ={u/G(x), v/x}, to produce the resolvent clause

[Animal (F(x)) ∨ ¬Kills(G(x), x)] .

This rule is called the binary resolution rule because it resolves exactly two literals.

Binary resolution alone is incomplete. Full resolution allows resolving any unifiable literals in clauses. Factoring removes redundant literals—identical ones in propositional logic, unifiable ones in first-order logic (with the unifier applied to the whole clause). Together, binary resolution and factoring form a complete inference procedure.

3.9.3 Example proofs

- Resolution proves that KB |= α by proving KB ∧ ¬α unsatisfiable, that is, by deriving the empty clause.
- The sentences in CNF are

  ¬American(x) ∨ ¬Weapon(y) ∨ ¬Sells(x, y, z) ∨ ¬Hostile(z) ∨ Criminal (x)

  ¬Missile(x) ∨ ¬Owns(Nono, x) ∨ Sells(West, x, Nono)

  ¬Enemy(x,America) ∨ Hostile(x)

  ¬Missile(x) ∨Weapon(x)

  Owns(Nono,M1) Missile(M1)

  American(West) Enemy(Nono,America) .

- We also include the negated goal ¬Criminal (West).

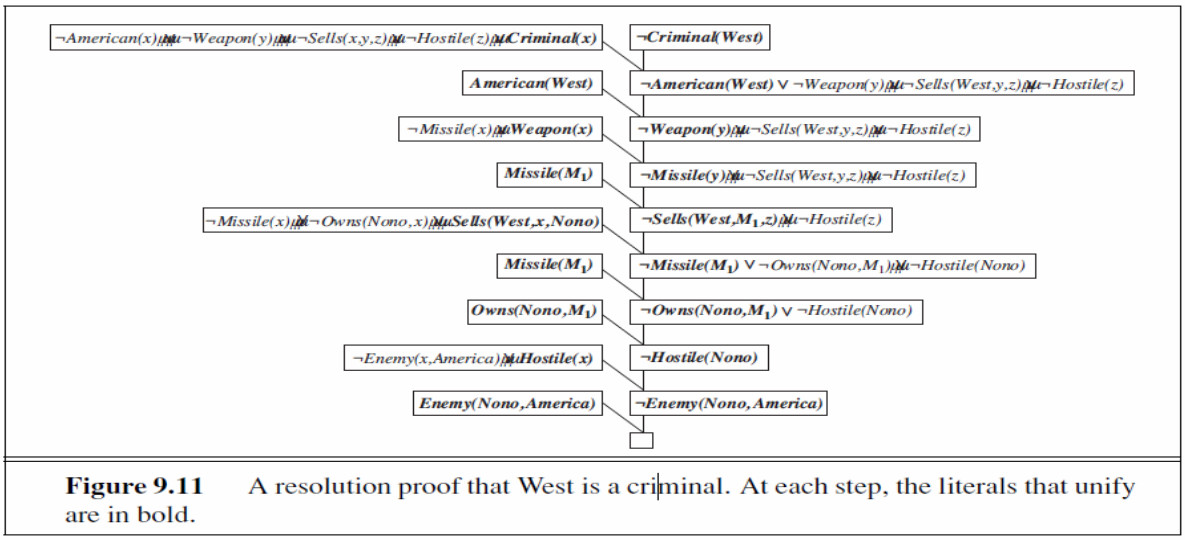


Figure 9.11 A resolution proof that West is a criminal. At each step, the literals that unify are in bold.

Step 1: Negated Goal

We want to prove:

Criminal(West)

So, we start by negating the goal:

¬Criminal(West)

This clause becomes the starting point of the resolution proof.

Step 2: Resolution Spine (Backward Chaining Style)

We now repeatedly resolve with clauses from the knowledge base (KB):

1.¬**Criminal(West)** resolves with

(¬American(x)∨¬Weapon(y)∨¬Sells(x,y,z)∨¬Hostile(z)∨Criminal(x))

→ gives: ¬American(West) ∨ ¬Weapon(y) ∨ ¬Sells(West, y, z) ∨ ¬Hostile(z).

2. ¬American(West) resolves with

American(West)

→ cancels out, leaving: ¬Weapon(y) ∨ ¬Sells(West, y, z) ∨ ¬Hostile(z).

3. ¬Weapon(y) resolves with

Missile(x)∨Weapon(x)

→ substitution y = M1 gives: ¬Missile(M1) ∨ ¬Sells(West, M1, z) ∨ ¬Hostile(z).

4. Missile(M1) from KB resolves with ¬Missile(M1):

→ result: ¬Sells(West, M1, z) ∨ ¬Hostile(z).

5. ¬Sells(West,M1,z)resolves with

¬Missile(x)∨¬Owns(Nono,x)∨¬Sells(West,x,z)

→ substitution x = M1 gives: ¬Owns(Nono, M1) ∨ ¬Hostile(Nono).

6. Owns(Nono, M1) (from KB) cancels ¬Owns(Nono, M1):

→leaves ¬Hostile(Nono).

7.¬Hostile(Nono) resolves with

¬Enemy(x,America)∨Hostile(x)

→ gives ¬Enemy(Nono, America).

8.Enemy(Nono, America) (from KB) cancels ¬Enemy(Nono, America):

→ Empty clause reached.

Step 3: Conclusion

Since the empty clause is derived, the negated goal ¬Criminal(West) leads to contradiction.

Therefore, Criminal(West) is true.

Our second example makes use of Skolemization and involves clauses that are not definite clauses. This results in a somewhat more complex proof structure. In English, the problem is as follows:

Everyone who loves all animals is loved by someone.

Anyone who kills an animal is loved by no one.

Jack loves all animals.

Either Jack or Curiosity killed the cat, who is named Tuna.

Did Curiosity kill the cat?

First, we express the original sentences, some background knowledge, and the negated goal G in first-order logic:

A. ∀ x [∀ y Animal (y) ⇒ Loves(x, y)] ⇒ [∃ y Loves(y, x)]

B. ∀ x [∃ z Animal (z) ∧ Kills(x, z)] ⇒ [∀ y ¬Loves(y, x)]

C. ∀ x Animal(x) ⇒ Loves(Jack, x)

D. Kills(Jack, Tuna) ∨ Kills(Curiosity, Tuna)

E. Cat(Tuna)

F. ∀ x Cat(x) ⇒ Animal (x)

¬G. ¬Kills(Curiosity, Tuna)

Now we apply the conversion procedure to convert each sentence to CNF:

A1. Animal(F(x)) ∨ Loves(G(x), x)

A2. ¬Loves(x, F(x)) ∨ Loves(G(x), x)

B. ¬Loves(y, x) ∨ ¬Animal (z) ∨ ¬Kills(x, z)

C. ¬Animal(x) ∨ Loves(Jack, x)

D. Kills(Jack, Tuna) ∨ Kills(Curiosity, Tuna)

E. Cat(Tuna)

F. ¬Cat(x) ∨ Animal (x)

¬G. ¬Kills(Curiosity, Tuna)

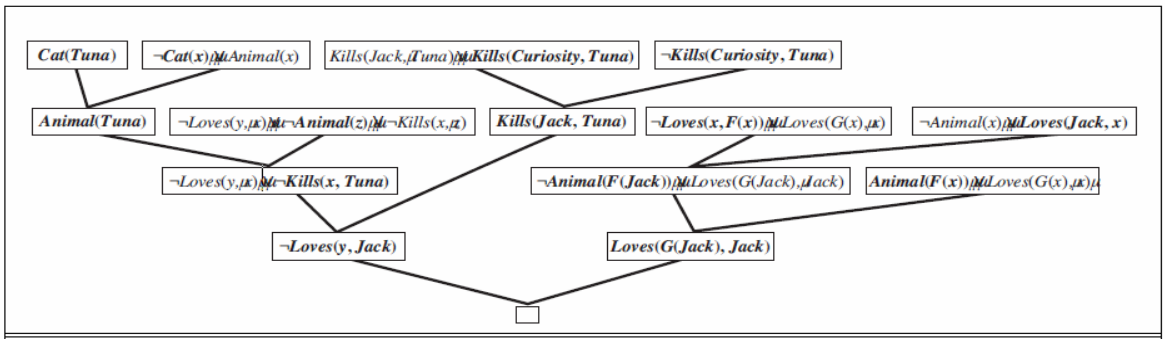


Figure 9.12 A resolution proof that Curiosity killed the cat. Notice the use of factoring in the derivation of the clause $Loves(G(Jack), Jack)$. Notice also in the upper right, the unification of $Loves(x, F(x))$ and $Loves(Jack, x)$ can only succeed after the variables have been standardized apart.

1. Facts introduced:

Cat(Tuna)

Animal(Tuna)

Kills(Jack, Tuna)

Kills(Curiosity, Tuna)

2. Rules (in clause form):

¬Cat(x) ∨ Animal(x)

¬Loves(y, x) ∨ ¬Animal(z) ∨ ¬Kills(x, z)

¬Animal(x) ∨ Loves(Jack, x)

¬Loves(x, F(x)) ∨ ¬Loves(G(x), x)

These encode statements about cats, animals, love, and killing.

3. Resolution steps:

- From Cat(Tuna) and ¬Cat(x) ∨ Animal(x) we infer Animal(Tuna).

- From Kills(Curiosity, Tuna) and rules about love/killing, we eventually derive conditions on Loves(...).
- Through unification of variables and factoring, we reduce the clauses.

4.Critical unification (upper right of the figure): Loves(x, F(x)) and Loves(Jack, x) → to unify, variables must be standardized apart (renamed).

- Example:
- Loves(x, F(x)) (with x)
- Loves(Jack,y)(with y)Only after renaming x → u, y → v, can we unify consistently.

5. Final derived clause:

Loves(G(Jack), Jack)

Then, combining with its negation (¬Loves(y, Jack)), we reach contradiction (empty clause).

To answer "Who killed the cat?" we negate the query ∃w Kills(w, Tuna) as ¬Kills(w, Tuna). Resolution can then provide variable bindings (e.g., {w/Curiosity}). However, resolution may give nonconstructive proofs (e.g., w could be Jack or Curiosity).

To avoid this, we can either restrict variable bindings to one per proof with backtracking, or add an answer literal (¬Kills(w, Tuna) ∨ Answer(w)), so the proof yields concrete answers like Answer(Curiosity).

3.9.4 Completeness of resolution

Resolution is refutation-complete: if a set of clauses is unsatisfiable, resolution will eventually derive a contradiction. The proof uses

- (1) Herbrand's theorem to reduce to ground instances,
- (2) completeness of propositional resolution for ground sentences, and
- (3) the lifting lemma to extend ground proofs back to first-order logic.

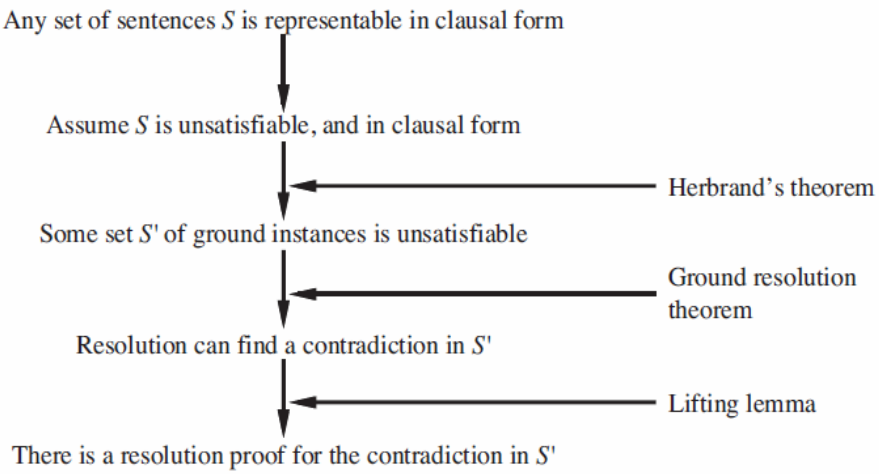


Figure 9.13 Structure of a completeness proof for resolution.

To carry out the first step, we need three new concepts:

• **Herbrand universe:** If S is a set of clauses, then H_S, the Herbrand universe of S, is the set of all ground terms constructable from the following:

a. The function symbols in S, if any.

b. The constant symbols in S, if any; if none, then the constant symbol A.

For example, if S contains just the clause ¬P(x, F(x,A))∨¬Q(x,A)∨R(x,B), then

H_S is the following infinite set of ground terms:

{A,B, F(A,A), F(A,B), F(B,A), F(B,B), F(A,F(A,A)), . . .}

Saturation: If S is a set of clauses and P is a set of ground terms, then P(S), the saturation of S with respect to P, is the set of all ground clauses obtained by applying all possible consistent substitutions of ground terms in P with variables in S.

Herbrand base: The saturation of a set S of clauses with respect to its Herbrand universe is called the Herbrand base of S, written as $H_S(S)$. For example, if S contains solely the clause just given, then $H_S(S)$ is the infinite set of clauses

{¬P(A, F(A,A)) ∨ ¬Q(A,A) ∨ R(A,B),

¬P(B,F(B,A)) ∨¬Q(B,A) ∨ R(B,B),

¬P(F(A,A), F(F(A,A),A)) ∨¬Q(F(A,A),A) ∨ R(F(A,A),B),

¬P(F(A,B), F(F(A,B),A)) ∨ ¬Q(F(A,B),A) ∨ R(F(A,B),B), . . . }

These definitions allow us to state a form of Herbrand's theorem

3.9.5 Equality

Axiomatization of equality in first-order logic:

Reflexive:

Symmetric:

Transitive:

Substitution for predicates (example for predicate):

Substitution for functions (example for function):

The first three handle equality as a relation, while the last two handle replacing equals inside predicates and functions.

1. Axiomatization (basic method)

- Add axioms: Reflexive, Symmetric, Transitive, Substitution.
- Works, but generates many useless conclusions → inefficient.

2. Inference rules (more efficient)

Demodulation:

- Rule: $x=y, \alpha \Rightarrow \alpha[y/x]$
- Always replace x with y (directional).
- Example:
- Father(Father(x))=PaternalGrandfather(x)

  Birthdate(Father(Father(Bella)),1926)

  Birthdate(PaternalGrandfather(Bella),1926)

Axioms = simple but inefficient.

Demodulation/Paramodulation = inference-rule based, complete.

Equational unification = most efficient, built into unification.

3.9.6 Resolution strategies

1. Unit Preference / Unit Resolution

- Prefer resolutions involving unit clauses (single literals).
- Produces shorter clauses → faster proofs.
- Unit resolution = every step must involve a unit clause.
- Complete only for Horn clauses.

2. Set of Support

- Require each resolution to involve at least one clause from a special set of support.

- Resolvent added back into set.
- If set is chosen well → efficient, goal-directed proofs.
- Complete if KB – S is satisfiable.

3. Input Resolution

- Every step uses one original input sentence (KB or query).
- Proof shape: "single spine."
- Complete for Horn KBs, incomplete otherwise.
- Linear resolution (generalization): allow ancestor clauses too → complete.

4. Subsumption

- Eliminate clauses that are subsumed (more specific than existing ones).
- Example: if P(x) exists, drop P(A) or P(A)∨Q(B).
- Keeps KB and search space small.

Unit preference → shorter clauses.

Set of support → goal-directed, efficient.

Input/Linear resolution → smaller proof space.

Subsumption → prune redundant clauses.